

0、【课程网页】

<https://c.d2l.ai/stanford-cs329p/v>

斯坦福2021秋季新课：实用机器学习

- 课程地址：<https://c.d2l.ai/stanford-cs329p>
- 关注机器学习落地的技术，是本课的补充

- 基础建模
 - 数据收集
 - 数据处理
 - 模型验证
 - 模型融合
- 不正确的假设
 - Covariate/
concept/label 偏
移
 - IID以外的数据（时
序、图）
- 性能调优
 - AutoML
 - 模型蒸馏
 - 多模块数据
- 模型之外
 - 模型部署
 - 公平

【目录】

0、【课程网页】	1
一、 数据	4
1.1 数据获取	4
1.2 网页数据获取	5
1.3 数据标注 data labeling	6
二、 数据 ii	9
2.1 探索性数据分析 eda-exploratory data analysis	9
2.2 数据清理	13
2.3 特征工程	17
2.4 数据总结	19
三、 机器学习模型	19
3.1 机器学习介绍	19
3.2 决策树模型	20
3.3 线性模型	22
3.4 神经网络	25
3.5 卷积神经网络 CNN	26
3.6 循环神经网络 RNN	28
四、 模型评估指标	30
4.1 模型评估	30
4.2 欠拟合和过拟合	34
4.3 模型验证 model validation	36
五、 模型融合	38

5.1 偏差和方差	38
5.2 Bagging	40
5.3 Boosting	42
5.4 Stacking	44
六、 调参	46
6.1 模型调参	46
6.2 超参数优化 HPO	47
6.3 网络架构搜索 (NAS algorithms)	51
七、 深度神经网络共用设计模式	54
7.1 批量和层归一化	54
7.2 残差连接	57
7.3 注意力机制	57
八、 迁移学习	57
8.1 迁移学习	57
8.2 微调在计算机视觉的应用 (fine-tuning in cv)	58
8.3 微调在自然语言处理的应用 (fine-tuning in NLP)	60

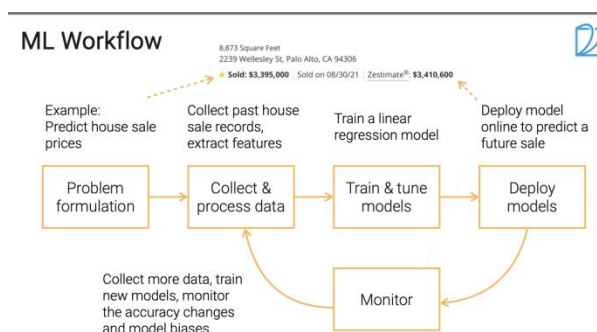
【完结撒花】

机器学习

工业界应用

- 健康：传感器
- 零售：预测销量
- 金融：欺诈检测
- 自动化：无人驾驶

机器学习流程



问题（房价预测的是回归问题）——收集和预处理数据（收集所有房子的数据，抽取特征）——训练模型和调参（用简单模型测试数据好坏）——模型部署——关注模型性能——部署后得到新的数据——迭代（更新数据）

【挑战】

- 1、把问题变成机器学习问题（比如无人车问题、self-service 商场）
- 2、数据：高质量的数据很难，需要做大量的清洗、标注；涉及到隐私问题

- 3、模型训练：模型越来越复杂和昂贵，对数据的需求越来越多
- 4、部署模型：gpu、实时性需要对模型的大小进行严格控制
- 5、监控模型：长期过程：数据分布的变化、算法替代人进行决策涉及公平性问题

【角色】

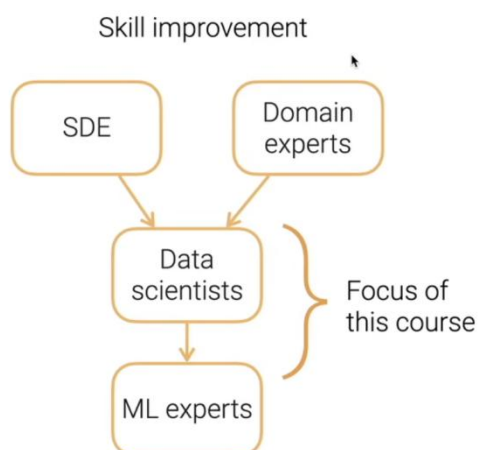
领域专家：具有商业洞察，对产品和商业有非常好的理解，知道哪些数据比较重要和他们在哪些地方，知道部署一个机器学习模型对产品有多大的真实影响

数据科学家： 全站工程师： 数据挖掘-训练模型-上线模型

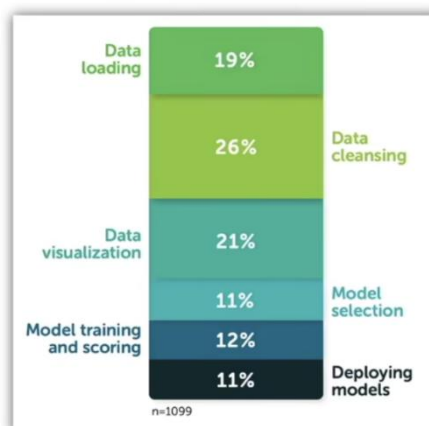
机器学习专家：能够对标重要问题（如对搜索引擎来讲，广告占了90%的营收），能够对产品的特点，比如需要什么样的模型，大概是什么样的性质（比如实时性特别好/对图片/文字处理的特别好），对模型做定制化

工程师 sde：管理模型训练和资源（数据训练维护部署）

Roles



How data scientists spent their time (source: Anaconda survey 2020)



数据

- 收集/处理数据
- 数据偏移
- 数据市场图

训练模型

- 模型验证/融合/调整超参数
- transfer-迁移学习
- 多模态（文本、图片等不同数据源的融合）

模型部署 (deploy)

-模型部署

-蒸馏 distillation (训练大的模型, 做的小一点小一点)

监控

-公平

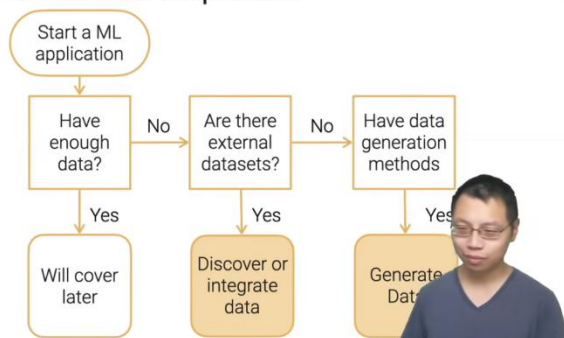
-可解释性 (理解模型在干什么事情)

一、 Basic ML Modeling

一、 数据

1.1 数据获取

Flow chart for data acquisition



【数据的发现】

—找已有的数据集

—超参数调节的算法在不同类型数据集上的性能如何

—大的数据集给深的神经网络

—收集新的数据

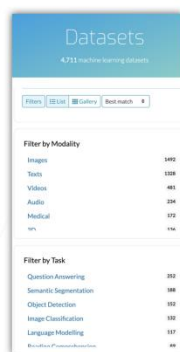
数据来源: 网上爬/采集数据

找数据

https://c.d2l.ai/stanford-cs329p/_static/pdfs/cs329p_slides_1_3.pdf

Where to Find Datasets

- [Paperswithcodes Datasets](#): academic datasets with leaderboard
- [Kaggle Datasets](#): ML datasets uploaded by data scientists
- [Google Dataset search](#): search datasets in the Web
- Various toolkits datasets: [tensorflow](#), [huggingface](#)
- Various conference/company ML competitions
- [Open Data on AWS](#): 100+ large-scale raw data
- Data lakes in your own organization



Datasets comparison

	Pros	Cons
Academic datasets	Clean, proper difficulty	Limited choices, too simplified, usually small scale
Competition datasets	Closer to real ML applications	Still simplified, and only available for hot topics
Raw Data	Great flexibility	Needs a lot of effort process

- You often need to deal with raw data in industrial settings
- Data curation can be a big projection involving multiple teams. Processing pipeline, storage, legal issue, privacy,...



数据融合

—— 需要把不同表的数据融合起来

—— join 合并起来

Inner join 只把两个表都有的信息合并起来

Left join 把所有的 id 存下来

—— 关键：确认 join 的锚点、处理缺失行、重复 col 去掉、类重复中数值冲突（单位、值）

生成合成的 synthetic 数据

-使用 GANs: 人脸、商家演示

-数据增强: 图片、文本增强

1.2 网页数据获取

爬: 把所有网页拿下来存好

数据抓取: 返回的是某一个感兴趣的网站的数据提取出来, 最后可能变成一个 table

```

1 from selenium import webdriver
2
3 chrome_options = webdriver.ChromeOptions()
4 chrome_options.headless = True
5 chrome = webdriver.Chrome(
6     chrome_options=chrome_options)
7
8 page = chrome.get(url)

```

Webdriver-可以认为是 chrome 的后台

把 chrome 的属性拿出来，把 headless 设成真——不需要图形界面了

在 python 里创建一个实例

创建之后通过 get 一个属性把 url 刚进去，就会返回 html 的配置

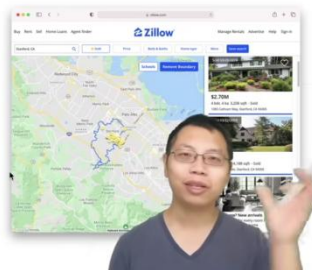
——很像是：你真打开一个 chrome 浏览器，把一个 url 放进去，然后返回一个网页然后把他保存下来

——大量的 ip

【实例】-房价预测

Case study - house price prediction

- Query houses sold in near Stanford
 - <https://www.zillow.com/stanford-ca/sold/>
 - <https://www.zillow.com/stanford-ca/sold/2-p/>
 - ...
- You can replace the city and state in the URL for other places



把房子 id 从网页里面抓取出来

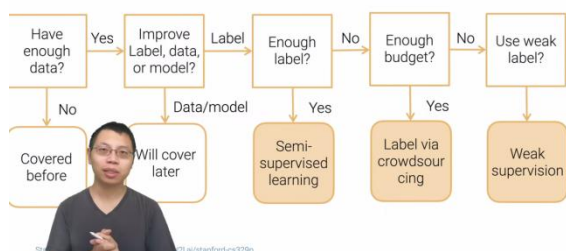
Beautifulsoup-python 的包-解析 html

Link-把所有 html 里面的 a link 那个元素找出来

把 link 切开拿到独特的 id-把 id 放到模版里

1.3 数据标注 data labeling

Flow Chart for Data Labelling



有足够多的标注——可以考虑半监督学习

有钱——抓人来标注数据

又没有钱又没有足够多的标注——从数据里面提出弱一点的标注，弱监督学习

半监督学习：主要解决一小部分数据有标注，但还有很多很多没有标注的数据

——半监督学习假设 (semi-supervised learning SSL)

连续性假设：如果一个样本和另一个样本特征是相似的，那么这两个样本很可能有相同的标号

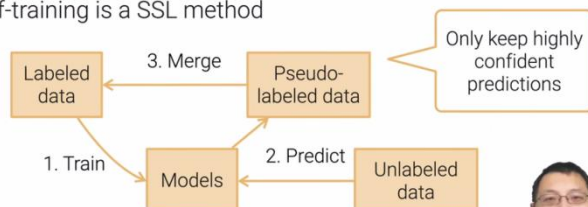
聚类的假设：数据在内在有比较好的结构，可以假设一个类里面的数据可能会有相同的标号

流行性假设：数据本质上是在一个低维的流形上分布，即数据内在的复杂性远远比看到的维度要低，可以通过降维来获取更干净的数据

半监督学习经常用到一个算法叫自学习

Self-training

- Self-training is a SSL method



- We can use expensive models

- Deep neural networks, model ensemble/bagging

假设有一小部分已经标好的数据，训练一个模型，用这个模型对没有标号的数据进行预测，把伪标号的数据和已经标号的数据合并起来再一起进行预测

——可以把比较确性的标号留下来，分类1类98%

——不用线上部署，可以考虑比较贵的模型，增加了计算复杂度很有可能带来性能上的提升

——可以用比较深的神经网络、模型融合

找人来标数据

Label through Crowdsourcing



- ImageNet labeled millions of images through Amazon Mechanical Turk. It took several years and millions dollars to build
- According to Amazon SageMaker Ground Truth, the estimated price of using Amazon Mechanical Turk:

Image/text classification	\$0.012 per label
Bounding box	\$0.024 per box
Semantic segmentation	\$0.84 per image



Stanford CS 239D / 239D1 Fall11 - <https://r.r71.ai/stanford-cs239d>

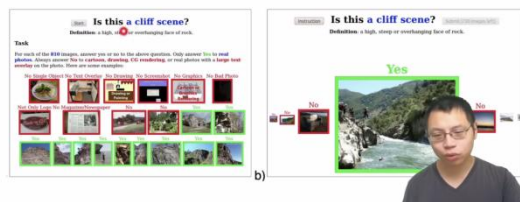
众包来标数据的挑战:

- 1、任务需要设计的比较简单，有指示
- 2、成本：整个数据要生成多少任务*每个任务时间
- 3、质量控制

——帮助:

1 User interaction

- The user instruction and task used by the MIT Place365 dataset



是还是不是

2 减少任务的数目——主动学习 (像 ssl)

Reduce #tasks: Active Learning



- Focus on same scenario as SSL but with human intervention
 - Select the most "interesting" unlabeled data to labelers
- **Uncertainty sampling** chooses an example whose prediction is most uncertain
 - The highest class prediction score is close to random ($1/n$)
- Similar to self-training we can use expensive models
 - **Query-by-committee** trains multiple models and performs majority voting

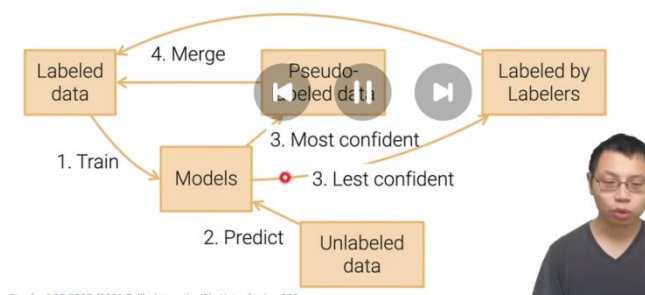


对最不确定性的样本让人来看是什么意思

主动学习+自训练

Active Learning + Self-training

- These two methods are often used together



3 数据质量控制

将每个图片每个任务发给多个标注工，投票——但很贵
把结果不是很确信的发给多个人

Weak supervision——弱监督学习

半自动生成标号

总结规律

Weak Supervision

- Semi-automatically generate labels
 - Less accurate than manual ones, but good enough for training
- **Data programming**: heuristic programs to assign labels
 - Keyword search, pattern matching, third-party models
 - E.g. rules to check if YouTube comments are spam or ham

```
1 def check_out(x):  
2     return SPAM if "check out" in x.lower() else ABSTAIN  
3  
4 def sentiment(x):  
5     return HAM if sentiment_polarity(x) > 0.9 else ABSTAIN  
6  
7 def short_comment(x):  
8     return HAM if len(x.split()) < 5 else ABSTAIN  
9  
10 Adapted from snorkel.org
```

Stanford CS 329P (2021 Fall) - <https://c.d2l.ai/stanford-cs329p/>

【总结】

拿到更多标注的方法

- 1、自训练
- 2、众包
- 3、启发式规则

也可以选择无监督/自监督的算法

二、数据 ii

2.1 探索性数据分析 eda-exploratory data analysis

Seaborn

https://c.d2l.ai/stanford-cs329p/_static/notebooks/cs329p_notebook_eda.slides.html#/1

- 1、Data.shape
- 2、一列里面有大于 30% 缺数据丢掉

We drop columns that at least 30% values are null to simplify our EDA.

```
In [5]: null_sum = data.isnull().sum()
data.columns[null_sum < len(data) * 0.3] # columns will keep
executed in 8.45s, finished 11:07:13 2021-09-19

Out[5]: Index(['Id', 'Address', 'Sold Price', 'Sold On', 'Summary', 'Type',
'Year built', 'Heating', 'Cooling', 'Parking', 'Bedrooms', 'Bathrooms',
'Total interior livable area', 'Total spaces', 'Garage spaces',
'Home type', 'Region', 'Elementary school', 'Elementary School Score',
'Elementary School Distance', 'High School', 'High School Score',
'High School Distance', 'Heating features', 'Parking features',
'Lot size', 'Parcel number', 'Tax assessed value', 'Annual tax amount',
'Listed On', 'Listed Price', 'Zip'],
dtype='object')

In [6]: data.drop(columns=data.columns[null_sum > len(data) * 0.3], inplace=True)
executed in 65ms, finished 11:07:13 2021-09-19
```

- 4、看数据类型是否正确 data.dtype
- 5、数据转换

金额、面积转成 float

Convert currency from string format such as \$1,000,000 to float.

```
In [8]: currency = ['Sold Price', 'Listed Price', 'Tax assessed value', 'Annual tax amount']
for c in currency:
    data[c] = data[c].replace(
        r'[$,-]', '', regex=True).replace(
        r'^\s*$', np.nan, regex=True).astype(float)
executed in 2.29s, finished 11:07:15 2021-09-19
```

Also convert areas from string format such as 1000 sqft and 1 Acres to float as well.

```
In [9]: areas = ['Total interior livable area', 'Lot size']
for c in areas:
    acres = data[c].str.contains('Acres') == True
    col = data[c].replace(r'\b sqft\b\b Acres\b\b', '', regex=True).astype(float)
    col[acres] = 43560
    data[c] = col
executed in 837ms, finished 11:07:16 2021-09-19
```

- 6、data.describe()

Now we can check values of the numerical columns. You could see the min and max values for several columns do not make sense.

```
In [10]: data.describe()
executed in 227ms, finished 11:07:16 2021-09-19

Out[10]:
```

	Id	Sold Price	Bathrooms	Total interior livable area	Total spaces	Garage spaces	Elementary School Score	Elementary School Distance	High
count	1.649440e+05	1.648590e+05	141791.000000	1.465450e+05	156738.000000	156736.000000	145676.000000	146288.000000	144511.000000
mean	2.791434e+08	1.194842e+06	2.303087	3.182221e+03	1.706044	1.607614	5.654892	1.260918	6.086485
std	6.424318e+08	3.336365e+06	1.646634	4.609881e+05	28.802242	28.782370	2.098547	2.855889	2.033375
min	7.387732e+06	1.000000e+00	0.000000	1.000000e+00	-26.000000	-26.000000	1.000000	1.000000	1.000000
25%	1.913563e+07	4.350000e+05	2.000000	1.170000e+03	0.000000	0.000000	4.000000	4.000000	5.000000
50%	2.059865e+07	8.050000e+05	2.000000	1.558000e+03	1.000000	1.000000	6.000000	6.000000	6.000000
75%	8.923942e+07	1.370000e+06	3.000000	2.144000e+03	2.000000	2.000000	7.000000	7.000000	8.000000
max	2.147000e+09	8.660000e+08	256.000000	1.764164e+08	9999.000000	9999.000000	10.000000	10.000000	10.000000

- 7、处理异常数据

We filter out houses whose living areas are too small or too hard to simplify the visualization later.

```
In [11]: abnormal = (data[areas[1]] < 10) | (data[areas[1]] > 1e4)
data = data[~abnormal]
sum(abnormal)
```

executed in 71ms, finished 11:07:16 2021-09-19

Out[11]: 41000



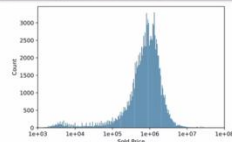
8、看数据特征分布

要预测的值-加了 log

Let's check the histogram of the 'Sold Price', which is the target we want to predict.

```
In [12]: ax = sns.histplot(np.log10(data['Sold Price']))
ax.set_xlim(1, 8)
ax.set_xticks(range(1, 9))
ax.set_xticklabels(['%0.1e' % a for a in 10**ax.get_xticks()])
```

executed in 1.08s, finished 11:07:17 2021-09-19



A house has different types. Here are the top types:

```
In [13]: data['Type'].value_counts()[0:20]
```

executed in 14ms, finished 11:07:17 2021-09-19

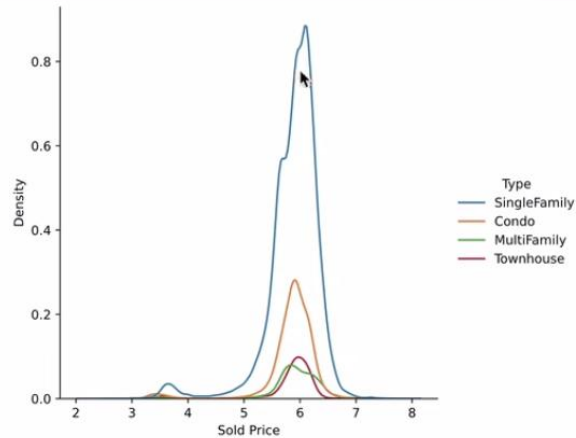
```
Out[13]: SingleFamily      74318
Condo                    18749
MultiFamily              6586
VacantLand                6199
Townhouse                 5846
Unknown                  5390
MobileManufactured       2588
Apartment                1416
Cooperative               161
Residential Lot           75
Single Family             69
Single Family Lot         56
Acreage                   48
2 Story                   39
3 Story                   25
Hi-Rise (9+), Luxury      21
RESIDENTIAL               19
Duplex                    19
Condominium               19
Mid-Rise (4-8)            17
Name: Type, dtype: int64
```

不同类型房子的价格

Price density for different house types.

```
In [14]: types = data['Type'].isin(['SingleFamily', 'Condo', 'MultiFamily', 'Townhouse'])
sns.displot(pd.DataFrame({'Sold Price':np.log10(data[types]['Sold Price']),
                          'Type':data[types]['Type']}),
            x='Sold Price', hue='Type', kind='kde');
```

executed in 1.14s, finished 11:07:18 2021-09-19



单位面积的价格-boxplot 75%、均值、25%

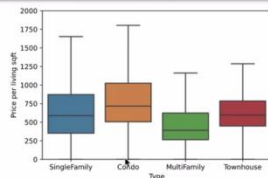
——直观表示不同分布之间的对比



Another important measurement is the sale price per living sqft. Let's check the differences between different house types.

```
In [15]: data['Price per living sqft'] = data['Sold Price'] / data['Total interior livable area']
ax = sns.boxplot(x='Type', y='Price per living sqft', data=data[types], fliersize=0)
ax.set_ylim([0, 2000]);
```

executed in 276ms, finished 11:07:19 2021-09-19

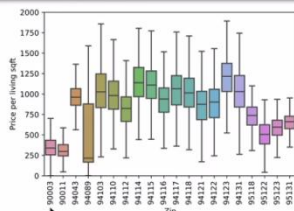


邮政编码（地域）

We know the location affect the price. Let's check the price for the top 20 zip codes.

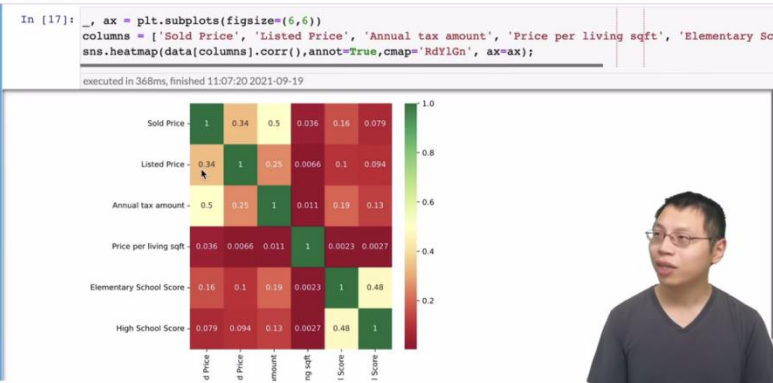
```
In [16]: d = data[data['Zip'].isin(data['Zip'].value_counts()[1:20].keys())]
ax = sns.boxplot(x='Zip', y='Price per living sqft', data=d, fliersize=0)
ax.set_ylim([0, 2000])
ax.set_xticklabels(ax.get_xticklabels(), rotation=90);
```

executed in 509ms, finished 11:07:19 2021-09-19



每个特征之间的关系——协方差

Last, we visualize the correlation matrix of several columns.



【总结】

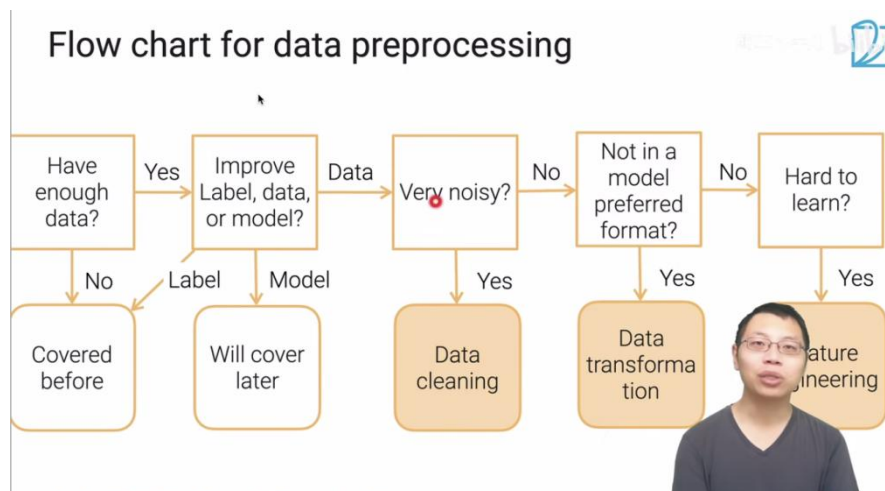
1 Summary

This notebook demonstrates the basic technologies for EDA, including

- Understanding column data types, values, and distributions
- Understanding the interactions between columns

We only explored a small aspect of the data. You are welcome to dive deep into more details.

2.2 数据清理



数据清理

数据变换

特征工程——数据对模型训练不友好

数据的错误

Data Errors

- Data often have errors - the mismatch with ground truth
 - e.g. missing, erroneous or extreme values
- Good ML models are resilient to errors
 - The training often converges, but the accuracy is degraded
- Deploying these models online may impact the quality of the new collected data

常见数据错误

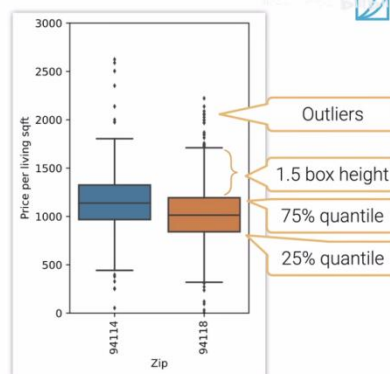
- 异常值
- 违背了规则
- 违反了语法上的限制

Types of Data Errors

- **Outliers**: data values that deviate from the distribution of values in the column
- **Rule violations**: violate integrity constraints such as "Not Null" and "Must be unique"
- **Pattern violations**: violate syntactic and semantic constraints such as alignment, formatting, misspelling



Outlier Detection



Rule-based Detection



- **Functional dependencies:** $x \rightarrow y$ means a value x determines a unique value y
 - E.g. zip code $\rightarrow$ state, EIN $\rightarrow$ company name
- **Denial constraints:** a more flexible first-order logic formalism
 - Phone number is not empty if vendor has an EIN
 - If two captures of the same animal indicated by the same tag number, then the first one must be marked as original



功能性的依赖

指定的限制（规则的检测）

Pattern-based Detection

- **Syntactic patterns**, e.g.
 - eng $\rightarrow$ English
 - Map a column to the most prominent data type and identify values do not fit
- **Semantic patterns**, e.g.
 - Add rules through knowledge graph
 - Values in column "Country" need have capitals, so a value "Stanford" is invalid

模式的检测

语言

知识图谱

【总结】

Summary



- Types of data errors: outliers, rule violations, pattern violations
- Detect errors and fix them by: model distribution, integrity constraints (functional dependencies, denial constraints), syntactic/semantic patterns
- Multiple tools exist to help data cleaning
 - Graphic interface for interactive cleaning
 - Automatically detect and fix



2.3 数据变换

Data Transformation



- ML algorithms prefer well defined fixed length, well-conditioned, nicely distributed input
- Next, data transformation methods for different data types



实数值特征常用的普通化手段

- 1、把一个列里面的最小值和最大值限定在一个固定的区间里-做了一个线性变化
 - 2、标准化
 - 3、使得所有值在 1 和-1 之间
 - 4、log 变换: log 上面的加减相当于原始上面的乘除
- 1000 块钱对不同房价的房子意义不同, 基于相对值 (百分比) 来计算

Normalization for Real Value Columns



- Normalization makes training more stable

Min-max normalization: linearly map to a new min a and max b	$x'_i = \frac{x_i - \min_{\mathbf{x}}}{\max_{\mathbf{x}} - \min_{\mathbf{x}}} (b - a) + a$
Z-score normalization: 0 mean, 1 standard deviation	$x'_i = \frac{x_i - \text{mean}(\mathbf{x})}{\text{std}(\mathbf{x})}$
Decimal scaling	$x'_i = x_i / 10^j \quad \text{smallest } j \text{ s.t. } \max(x'_i) < 1$
Log scaling	$x'_i = \log(x_i)$

图片转换

- 把图片尺寸变小
- 机器学习对于低分类的图片做的挺好的
- 注意图片的质量-会导致精度的下降
- 图片降维

Image Transformations



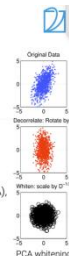
Image Transformations

- Our previous web scraping will scrape 15 TB images for a year
 - 5 millions houses sold in US per year, ~20 images/house, ~153KB per image, ~1041x732 resolution
- cropping, downsampling, compression
 - Save storage cost, faster loading at training
 - At ~320x224 resolution, 15 TB -> 1.4TB
 - ML is good at low-resolution images
 - Be aware of lossy compression
 - Medium (80%-90%) jpeg compression may lead to 1% acc drop in ImageNet



CIFAR-10

- Image whitening
 - Generalized normalization of vector values
 - Pixels in local neighborhood are highly correlated.
 - Whitening removes redundancy through linear transformations
 - Vector x has mean 0 and covariance estimate Σ
 - $y = Wx$, st $W^T W = \Sigma^{-1}$, y has unit diagonal covariance
 - Common choices of whitening matrix: Eigen-system of Σ (PCA), $\Sigma^{-\frac{1}{2}}$ (ZCA).
 - Model converges faster with whitened image input
 - Especially for unsupervised learning, e.g. GAN



Stanford CS 329P (2021 Fall) - <https://cs231n.stanford.edu/2021/>

视频转换

- 切到你最感兴趣的时间, 别的都不要

Video Transformations

- Input variability high
 - Average video length: Movies ~2h, YouTube videos ~11min, Tiktok short videos ~15sec
- Tractable ML problems with short video clips (<10sec)
 - Ideally each clip is a coherent event (e.g. a human action)
 - Semantic segmentation is extremely hard..
- Preprocessing to tradeoff storage, quality and loading speed
- Common practice: decode a playable video clip, sample a sequence of frames, compute spectrograms for audio
 - Easy to load to model, increased storage space

Video Transformations

- Input variability high
 - Average video length: Movies ~2h, YouTube videos ~11min, Tiktok short videos ~15sec
- Tractable ML problems with short video clips (<10sec)
 - Ideally each clip is a coherent event (e.g. a human action)
 - Semantic segmentation is extremely hard..
- Preprocessing to tradeoff storage, quality and loading speed
- Common practice: decode a playable video clip, sample a sequence of frames, compute spectrograms for audio
 - Easy to load to model, increased storage space

文本转换

-词根化、语法化

-词元化

Text Transformations

- Stemming and lemmatization: a word → a common base form
 - E.g. am, are, is → be car, cars, car's, cars' → car
 - Example: Topic modeling
- Tokenization: text string → a list of tokens (smallest unit to ML algorithms)
 - By word: `text.split(' ')`
 - By char: `text.split('')`
 - By subwords:
 - e.g. "a new gpu" → "a", "new", "gp", "##u", "t"
 - Custom vocabulary learned from the text corpus (Unigram, WordPiece)

总结

平衡数据的大小、质量和存储速度

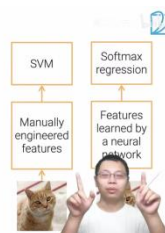
Summary

- Data transformation transfers data into formats preferred by ML algorithms, and balance data size, quality, and loading speed
- Tabular: normalize real value features
- Images: cropping, downsampling, whitening
- Videos: clipping, sampling frames
- Text: stemming, lemmatization, tokenization

2.3 特征工程

Feature Engineering

- Machine learning algorithms prefer well defined fixed length input / output
- Feature engineering (FE) is the key to ML models before deep learning (DL)
 - In a computer vision task, people try various FE methods and then train a SVM model
- DL train deep neural networks to extract features
 - Features are relevant to the task.



神经网络去学习参数使得我能对他进行比较好的特征抽取

Tabular Data Features

- Tabular data are in the form of a table, feature columns of numeric / categorical / string type
- Int/float: directly use or bin to n unique int values
- Categorical data: one-hot encoding
 - Map rare categories into "Unknown"
- Date-time: a feature list such as
 - [year, month, day, day_of_year, week_of_year, day_of_week]
- Feature combination: Cartesian product of two feature groups
 - [cat, dog] x [male, female] -> [(cat, male), (cat, female), (dog, male), (dog, female)]



fish cat and dog unknown
cat → [0, 1, 0, 0, 0]
dog → [0, 0, 0, 1, 0]

实数数据：分成 n 个区间

分类数据：独热编码

时间：展开成年、月、日（年）、天、周、日（周）

将特征组合起来——也是变成独热编码

Text Features

- Represent text as token features
 - Bag of words (BoW) model
 - Limitations: needs careful vocabulary design, losing context for individual words
 - Word Embeddings (e.g. Word2vec):
 - Vectorizing words such that similar words are placed close together
 - Trained by predicting target word from context words
- Pre-trained universal language models (e.g. universal sentence encoder, BERT, GPT-3)
 - Giant transformer models
 - Trained with large amount of unannotated data
 - Usage: Text embedding; fine-tuning for downstream tasks

dog and cat and dinosaur
↓
fish cat and dog unknown
[0, 1, 2, 1, 1]



文本数据

-把每个词元用独热编码加起来

词嵌入——把一个词表示一个向量，这个向量之间是有语义信息

-用预训练好的大的语言模型（BERT、GPT-3）

每个词进去就会返回一个向量给你

Word2vec 用的是一个比较小的神经网络，这个用的是一个比较大的 transformer 模型

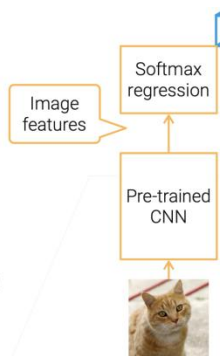
使用在很大的一些没有标注的数据上通过自训练自监督学习来进行训练

图片/视频特征

-用预训练好的神经网络模型

Image/Video Features

- Traditionally extract images by hand-craft features such as SIFT
- Now pre-trained deep neural networks are common used as feature extractor
 - ResNet: trained with ImageNet (image classification)
 - I3D: trained with Kinetics (action classification)
 - Many off-the-shelf models available



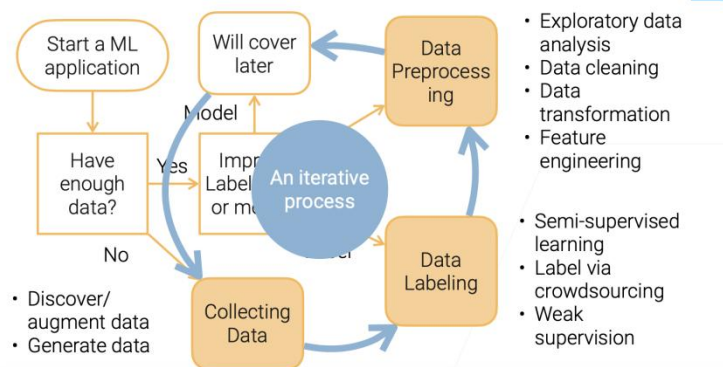
总结

Summary

- Features are representations of raw data that are relevant to the target task
- Feature engineering VS Feature learning
 - The latter is preferred if available (images/videos/audio/text)
 - Will cover more later in "transfer learning"

2.4 数据总结

Flow chart



三、机器学习模型

3.1 机器学习介绍

Types of ML Algorithms

Supervised	• Train on labeled data to predict labels
Semi-supervised	• Train on both labeled and unlabeled data, use models to infer labels for unlabeled data • E.g. self-training
Unsupervised	• Train on unlabeled data • E.g. clustering, density estimation
Reinforcement learning	• Use observations from the interaction with the environment to take actions to maximize reward

Types of ML Algorithms

Supervised	• We can design supervised training tasks for unlabeled data • Self-supervised learning: generate labels from data. E.g. word2vec, BERT • GAN: generating fake data with trivial label from unlabeled data.
Semi-supervised	
Unsupervised	• Training tasks can be different from how the model is evaluated / used.
Reinforcement learning	

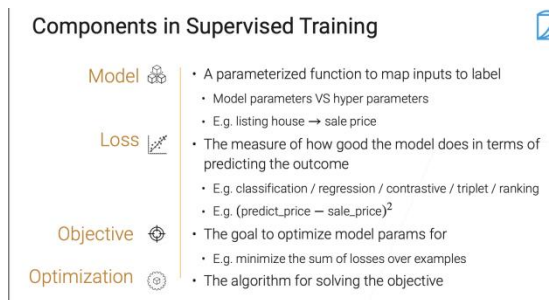
监督学习：在有标号的数据中去训练模型，任务就是预测这个标号

-自监督学习：也是可监督的学习，也产生标号，但他的标号来自数据本身（包括词嵌入、BERT、transformer 模型）

-半监督学习：自训练

无监督学习：无监督学习是没有标号的，任务也不是去预测一个标号，包括聚类算法（根据数据的相似性聚成不同的类）、把分布估计出来（GAN,生成对抗模型）

强化学习：模型和环境进行交互-机器人，最大化奖励，强化学习更像人类的学习方法



监督学习的重要成分

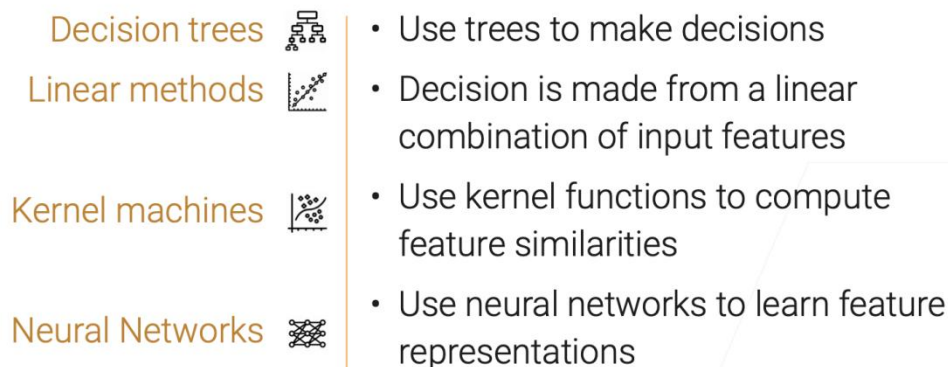
-模型本身：把输入转化为输出（标号）

-损失函数：衡量模型预测出来的值和真实值的差别（因为要知道模型尽量像真实值靠近，得告诉你定量该怎么算），如平方损失

-目标：在进行模型训练的时候要有一个目标，要去优化这个目标，优化什么样的函数，如要去优化使得在训练集合上所有预测的损失和最小

-优化：把模型里面没有指定的参数（可以学习的参数）填上值使得最小话损失

Types of Supervised Models



监督学习的模型

决策树：用书来做决定

线性模型：做决策的时候做决策的东西根据我输入的线性组合

核方法：用核方法来衡量两个样本特征之间的相似度

神经网络：用多层神经网络来学习特征

3.2 决策树模型

决策树

分类树、回归树

Decision Trees



- Explainable
- Handle both numerical and categorical features without preprocessing

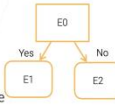
Building Decision Trees

- Use a top-down approach, starting from the root node with the set of all features
- At each parent node, pick a feature to split the examples

Feature selection criteria

- Maximize variance reduction for continuous target
- Maximize Information gain ($1 - \text{entropy}$) for categorical target
- Maximize Gini impurity $= 1 - \sum_{i=1}^n p_i^2$ for categorical target

- All examples are used for feature selection at each node



好处

- 1、可解释：模型训练出来之后，可以看叶子结点是什么东西，决策是怎么一步步做下来的（所以这个模型被大量用于银行业和保险业）
- 2、可以处理数值类和类别类的特征

坏处

- 1、非常不稳定：数据里面产生一定的噪音之后，可能整棵树构建出来的样子就不一样了（集成学习有一定的帮助）
- 2、数据特别复杂可以生成一个特别复杂的树导致过拟合（解决过拟合的办法是剪枝）
- 3、树并行起来在线上部署会难一些

一棵树不稳定，有很多办法让他变稳定，常见的办法是随机森林

随机森林（投票、平均）

-训练多个决策树提升稳定性

每棵树独立进行训练，训练出多棵树之后一起作用产生结果
分类就投票，回归的话实数值在每棵树上取平均

为什么叫随机

-随机产生的树，不是随机的话对稳定性提升有限

随机来自于两种情况

- 1、bagging：随机在训练集采样，而且替换地采样。拿到这个 bagging 出来的数据集之后就上面训练一棵树，一直重复训练 n 棵树为止
- 2、随机采样一些特征列，不用整个特征

Random Forest

- Train multiple decision trees to improve robustness
 - Trees are trained independently in **parallel**
 - Majority voting for classification, average for regression
- Where is the randomness from?
 - Bagging: randomly sample training examples with replacement
 - E.g. [1,2,3,4,5] $\rightarrow$ [1,2,2,3,4]
 - Randomly select a subset of features

另外一个用来提升树模型的办法——叫 boosting

Gradient boosting decision trees（求和）

训练多棵树顺序地完成（不再是独立的完成），这些树一起合成比较大的模型

过去训练所有树训练输出的和

在没有训练好的部分（残差-所有树预测的值和真实值的差）再训练一棵树。缩小真实值和预测值之间的差

-残差等价于预测值和真实值之间的均方误差做梯度并且取负号，负梯度就是梯度下降

【总结】

树模型是机器学习唯数不多的可解释模型，可以做分类也可以做回归

树模型的一大问题是不是那么的稳定，对数据的噪音比较敏感，可以通过很多棵树放在一起降低偏移和方差

随机森林-随机并行的训练多棵树，这些树合并起来做投票或者平均来做预测

Gradient boost-顺序地训练出一些树，每一棵新的树是之前那些树预测的不准的那一块部分去继续做拟合

树模型是工业界里面用的可能最多模型，因为训练算法比较简单，没有太多超参数可以调（不太需要调参，就不太容易出错，而且结果还不错，给你一个大概的想法，接下来有个不错的开头可以继续做下去，做下去的化左边可以提升数据的质量，右边可以换用一些更好的模型——所以树模型为什么在能够用的时候通常是第一选择）

Summary

- Decision tree: an explainable model for classification/regression
- Easy to train and tune, widely used in industry
- Sensitive to data
 - Ensemble can help (more on bagging and boosting latter)

3.3 线性模型

Linear Regression

- A simple house price prediction
 - Assume 3 features: $x_1 = \text{\#beds}$, $x_2 = \text{\#baths}$, $x_3 = \text{living sqft}$
 - The predicted price is $y = w_1x_1 + w_2x_2 + w_3x_3 + b$
 - Weights w_1, w_2, w_3 and bias b will be learnt from training data
- In general, given data $\mathbf{x} = [x_1, x_2, \dots, x_p]$, linear regression predicts
$$y = w_1x_1 + w_2x_2 + \dots + w_px_p + b = \langle \mathbf{w}, \mathbf{x} \rangle + b$$
- Trainable weights $\mathbf{w} = [w_1, w_2, \dots, w_p]$ and bias b



```
e: # weight w has shape (p, 1), [p, 1]
# bias b is a scalar
# data X has shape (p, 1)
y_hat = (x*w).sum() + b
```


线性模型做回归

Objective Function

- Collect n training examples $\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_1, \dots, \mathbf{x}_n]^T \in \mathbb{R}^{n \times p}$ with labels $\mathbf{y} = [y_1, \dots, y_n]^T \in \mathbb{R}^n$
 - E.g. house listings and sold prices
- Objective: minimize the mean square error (MSE)

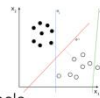
$$\begin{aligned} \mathbf{w}^*, \mathbf{b}^* &= \underset{\mathbf{w}, b}{\operatorname{argmin}} \mathcal{L}(\mathbf{X}, \mathbf{y}, \mathbf{w}, b) \\ &= \underset{\mathbf{w}, b}{\operatorname{argmin}} \frac{1}{n} \sum_{i=1}^n \|y_i - \langle \mathbf{x}_i, \mathbf{w} \rangle - b\|^2 \end{aligned}$$

- Exercise: write the closed form solution

线性模型做分类

Use linear regression for classification problem

- Regression: continuous output in $\mathbb{R}$
- Multi-class classification:
 - One-hot label $\mathbf{y} = [y_1, y_2, \dots, y_m]$ where $y_i = 1$ if $i = y$ otherwise 0
 - $\hat{\mathbf{y}} = \mathbf{o}$, where i -th output o_i is the confidence score for class i
 - Learn a linear model for each class $o_i = \langle \mathbf{x}, \mathbf{w}_i \rangle + b_i$
 - Minimize MSE loss $\frac{1}{m} \|\mathbf{o} - \mathbf{y}\|_2^2$
 - Predict label $\operatorname{argmax}_i \{o_i\}_{i=1}^m$
- Waste model capacity on pushing o_i near 0 for off labels



问题：分类关心的是真实类的置信度要够大，mse 带来的问题是要学习的太多了
解决办法-解决分类问题（线性模型+softmax 操作子）

Softmax Regression

- One-hot label $\mathbf{y} = [y_1, y_2, \dots, y_m]$ where $y_i = 1$ if $i = y$ otherwise 0
- $\hat{\mathbf{y}} = \operatorname{softmax}(\mathbf{o})$ where $\hat{y}_i = \frac{\exp(o_i)}{\sum_{j=1}^m \exp(o_j)}$
 - Turns confidence scores into probabilities (non-negative, sum to 1)
 - Ideally we want $\hat{\mathbf{y}} = \operatorname{one-hot}(\operatorname{argmax}_i o_i)$, softmax is a continuous approximate to that
 - Still a linear model, decision made on linear transformation of the input, as $\operatorname{argmax}_i \hat{y}_i = \operatorname{argmax}_i o_i$
- Cross-entropy loss between two distributions $\hat{\mathbf{y}}$ and $\mathbf{y}$
$$H(\mathbf{y}, \hat{\mathbf{y}}) = -\sum_i y_i \log(\hat{y}_i) = -\log \hat{y}_y$$
 - When label class is i , assigns less penalty on o_j as long as $o_j \ll o_i$
- Exercise: think about how to handle examples with multi labels?



```
0_exp = torch.exp(0)
partition = 0_exp.sum(
    1, keepdim=True)
Y = 0_exp / partition
```

交叉熵

——比较预测的概率和实际的概率（衡量两个概率之间的区别）

——衡量两个向量/分布之间的差别：yi 除了 1 都是 0，最后得到的结果是对样本真实预测的概率是多少，要去优化这个值（最小化）——使得在正确类别上预测的概率最大

求解线性模型

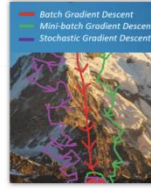
——可以通过显性解来求

——小批量随机梯度下降

Mini-batch Stochastic gradient descent (SGD)



- Train by mini-batch SGD (by various other ways as well)
 - $\mathbf{w}$ model param, b batch size, η_t learning rate at time t
 - Randomly initialize $\mathbf{w}_1$
 - Repeat $t = 1, 2, \dots$ until converge
 - Randomly samples $I_t \subset \{1, \dots, n\}$ with $|I_t| = b$
 - Update $\mathbf{w}_{t+1} = \mathbf{w}_t - \eta_t \nabla_{\mathbf{w}_t} \mathcal{L}(\mathbf{X}_{I_t}, \mathbf{y}_{I_t}, \mathbf{w}_t)$
- Pros: solve all objectives in this course except for trees
- Cons: sensitive to hyper-parameters b and η_t



converge 收敛：发现目标函数在每个样本上的平均损失不再下降/精度一些指标趋向平衡

随机采样一个小批量 I_t , 在这个上面把它当成当前的训练样本去算目标函数 L , 算出来之后对 $\mathbf{w}$ 进行求导, 把它的导数乘以学习率, 然后剪掉当前的权重, 得到新的权重

——这个公式后半段表示能够使得目标函数增加的那一个方向

——导数告诉我的是我沿着这个导数走的话, 把 $\mathbf{w}_t$ 改变的话, 能把我的目标函数值变大, 目标是变小, 所以加了个负号, 这个是告诉你方向, 到底走多远是学习率决定的

超参数: 批量大小 b , 学习率

Code

- Train a linear regression model with min-batch SGD
- Hyperparameters
 - batch_size
 - learning_rate
 - num_epochs

Full code at: http://d2l.ai/chapter_linear-networks/linear-regression-scratch.html

Stanford CS 329P (2021 Fall) - <https://c.d2l.ai/stanford-cs329p>

```
# `features` shape is (n, p), `labels` shape is (n, 1)
def data_iter(batch_size, features, labels):
    num_examples = len(features)
    indices = list(range(num_examples))
    random.shuffle(indices) # read examples at random
    for i in range(0, num_examples, batch_size):
        batch_indices = torch.tensor(
            indices[i:min(i + batch_size, num_examples)])
        yield features[batch_indices], labels[batch_indices]

w = torch.normal(0, 0.01, size=(p, 1), requires_grad=True)
b = torch.zeros(1, requires_grad=True)

for epoch in range(num_epochs):
    for X, y in data_iter(batch_size, features, labels):
        y_hat = X @ w + b
        loss = ((y_hat - y)**2 / 2).mean()
        loss.backward()
        for param in [w, b]:
            param -= learning_rate * param.grad
            param.grad.zero_()
```



总结

线性模型就是把输入进行线性加权和来得到预测

在线性回归里面用平均均方误差来作为损失函数

在 softmax 里面用交叉熵作为损失, 对每一个类进行线性预测, 用 softmax 操作子得到每一个类别的预测概率, 用来做多类分类的问题

小批量梯度下降两个模型都可以求解

3.4 神经网络

神经网络的作用可以认为把手工特征提取的部分换成神经网络

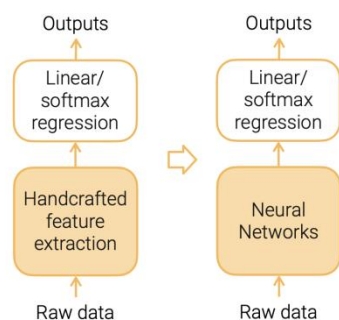
——之前是用人的知识对原始数据进行提取，现在是用神经网络对原始数据进行提取

——神经网络可以认为更懂机器学习，他提取出来的特征对后面的线性模型/softmax 回归更好一些

——要付出的代价：数量和计算量要大数量数量级

——选择不同的神经网络架构来得到更有效的抽取我们的特征：多层感知机、卷积神经网络、循环神经网络、变形金刚 transformer

Handcrafted Features → Learned Features



- NN usually requires more data and more computation
- NN architectures to model data structures
 - Multilayer perceptions
 - Convolutional neural networks
 - Recurrent neural networks
 - Attention mechanism
- Design NN to incorporate prior knowledge about the data

线性方法——》多层感知机 MLP

单层感知机最重要的一个东西叫做全连接层/线性层，这个层里面有可以学习的参数， w ，是一个 $m \times n$ 的东西， b 是一个偏移，长度为 m ，可以认为 n 是输入特征的维度， m 是要输出的那个向量的长度

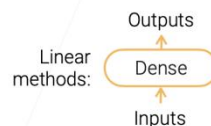
——线性回归：全连接层只有一个输出， $m=1$

——softmax 回归：有 m 个输出， m 是类别个数，再加上 softmax 操作子得到概率的输出

Linear Methods → Multilayer Perceptron (MLP)



- A **dense** (fully connected, or linear) layer has parameters $\mathbf{W} \in \mathbb{R}^{m \times n}$, $\mathbf{b} \in \mathbb{R}^m$, it computes output $\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b} \in \mathbb{R}^m$
- Linear regression: dense layer with 1 output
- Softmax regression: dense layer with m outputs + softmax



多层感知机 MLP

——得到非线性模型，使用多个全连接层

——简单的全连接层放在一起是没有用的， n 个线性操作叠加还是线性操作，所以需要加入非线性在里面，非线性就是激活函数，激活函数是按元素的非线性函数，对每个元素做 sigmoid/ReLU 操作

超参数：

1、隐藏层（除了最上面那层其余的 dense-例子 1 个、3 个）——为什么叫隐藏层，从输出的角度来看只能看到最上面那层的 dense，把多个隐藏层叠起来可以得到深一些的模式

2、每一个隐藏层输出的大小

Multilayer Perceptron (MLP)

- Activation is a elemental-wise non-linear function
 - $\text{sigmoid}(x) = \frac{1}{1 + \exp(-x)}$, $\text{ReLU}(x) = \max(x, 0)$
- It leads to non-linear models
- Stack multiple hidden layers (dense + activation) to get deeper models
- Hyper-parameters:
 - # hidden layers, # outputs of each hidden layer
- Universal approximation theorem

Stanford CS 329P (2021 Fall) - <https://cs.d2l.ai/stanford/cs329p>

Code

- MLP with 1 hidden layer
- Hyperparameter: num_hidden

```
def relu(X):
    return torch.max(X, 0)
```

```
W1 = nn.Parameter(torch.randn(num_inputs, num_hidden) * 0.01)
b1 = nn.Parameter(torch.zeros(num_hidden))
W2 = nn.Parameter(torch.randn(num_hidden, num_outputs) * 0.01)
b2 = nn.Parameter(torch.zeros(num_outputs))
```

```
H = relu(X @ W1 + b1)
Y = H @ W2 + b2
```

Full code: http://d2l.ai/chapter_multilayer_perceptrons/mlp-scratch.html

3.5 卷积神经网络 CNN

全连接层 dense layer——» 卷积层 convolution layer

例子

图片分类

——使用先验信息使得设计神经网络的时候可以把先验知识放进去

——在一个图片里识别一个物体的两个原则

1、做变换的时候不会变：找出带红帽子的人

2、本地性：找一个东西不需要看特别远的像素，像素跟周围像素相关性比较高，跟很远像素相关性低

Dense layer → Convolution layer

- Learn ImageNet (300x300 images with 1K classes) by a MLP with a single hidden layer with 10K outputs
 - It leads to 1 billion learnable parameters, that's too big!
 - Fully connected: an output is a weighted sum over all inputs
- Recognize objects in images
 - **Translation invariance**: similar output no matter where the object is
 - **Locality**: pixels are more related to near neighbors
- Build the prior knowledge into the model structure
 - Achieve same model capacity with less # params



卷积层利用了以上两个性质

——本地性：在卷积层里面一个输出只使用 $k \times k$ 的输入的一个大小窗口的图片

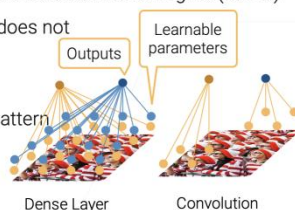
——平移不变性/变换不变性：如果有一个权重选好了的权重这四个点能识别一个东西，把这个东西移到别的地方一样能识别

——卷积层可学习参数个数不再和输入和输出大小相关，只和 k 的窗口相关

Convolution layer

- Locality: an output is computed from $k \times k$ input windows
- Translation invariant: outputs use the same $k \times k$ weights (kernel)
- # model params of a conv layer does not depend on input/output sizes

$$n \times m \rightarrow k \times k$$
- A kernel may learn to identify a pattern



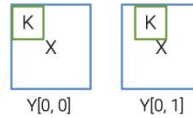
单通道卷积实现

Code



- Convolution with matrix input and matrix output (single channel)

```
# both input `X` and weight `K` are matrices
h, w = K.shape
Y = torch.zeros((X.shape[0] - h + 1, X.shape[1] - w + 1))
# stride = 1
for i in range(Y.shape[0]):
    for j in range(Y.shape[1]):
        Y[i, j] = (X[i : i+h, j : j+w] * K).sum()
```



- Exercise: implement multi-channel input / output convolution

Stanford CS 329P (2021 Fall) - <https://c.d2l.ai/stanford-cs329p>

Full code: http://d2l.ai/chapter_convolutional-neural-networks/conv-layer.html

卷积层对输入的位置非常敏感

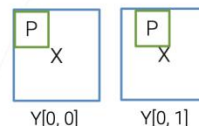
- 实际看到的图片里面一个物体不会总出现在一个位置
- 需要一定的对位置的鲁棒性
- 用 poolinglayer 池化层
- 计算 $k \times k$ 窗口的均值/最大值，均值叫平均汇聚，最大值叫最大汇聚
- 如果用的是最大，可以认为对卷积输入的 k 个像素的偏移没那么敏感

Pooling Layer



- Convolution is sensitive to location
 - A translation/rotation of a pattern in the input results similar changes of a pattern in the output
- A pooling layer computes mean/max in windows of size $k \times k$

```
# h, w: pooling window height and width
# mode: max or avg
Y = torch.zeros((X.shape[0] - h + 1, X.shape[1] - w + 1))
for i in range(Y.shape[0]):
    for j in range(Y.shape[1]):
        if mode == 'max':
            Y[i, j] = X[i : i+h, j : j+w].max()
        elif mode == 'avg':
            Y[i, j] = X[i : i+h, j : j+w].mean()
```



Full code: http://d2l.ai/chapter_convolutional-neural-networks/pooling.html

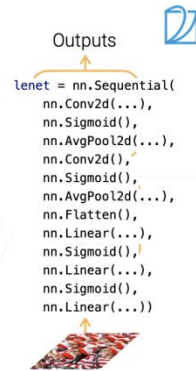
Stanford CS 329P (2021 Fall) - <https://c.d2l.ai/stanford-cs329p>

卷积神经网络 (CNN)

- 是一个神经网络，但是把各种卷积层堆起来，用卷积层来抽取图片里面的空间信息，只要满足本地性和变换不变性的性质，都可以用卷积层来做
- 激活层用在每个卷积层之后，卷积层可以认为是一个特殊的全连接层，限制住了一些权重为 0，只有一小部分的 w 可以非 0
- 卷积层对位置很敏感，用池化层来得到一些对位置没那么敏感的输出

Convolutional Neural Networks (CNN)

- Stacking convolution layers to extract features
 - Activation is applied after each convolution layer
 - Using pooling to reduce location sensitivity
- Modern CNNs are deep neural network with various hyper-parameters and layer connections (AlexNet, VGG, Inception, ResNet, MobileNet)



3.6 循环神经网络 RNN

全连接层——» 循环神经网络

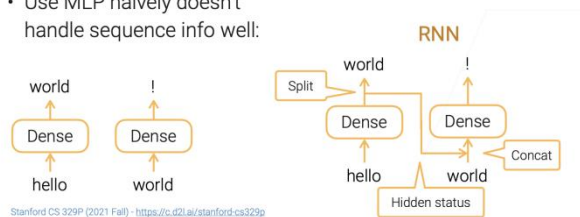
语言模型：预测下一个词

——核心思想：怎么把一个变长的序列表示成一个固定长度的向量

隐藏状态包括了过去时间所有的信息

Dense layer → Recurrent networks

- Language model: predict the next word
 - hello → world hello world → !
- Use MLP naively doesn't handle sequence info well:



具体做法

rnn 表示当前时刻输入 x_t 会得到一个输出 y_t ，输出复制一份得到隐藏状态 h_t ， h_t 本质上是和 y_t 是一个东西， h_t 作为隐藏状态放到下一个时刻，同样也拿到了上个时刻 h_{t-1} 的输出。计算：拿到的 h_{t-1} 对他做一个可学习的参数 whh 再加上当前的输入 x_t ，加上偏移，在做一个激活函数

常用的 RNN 层包括：LSTM、GRU

——里面做一些很细致的控制，信息流是怎么样流的

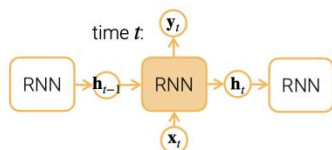
——忘掉输入：在计算 h_t 输出的时候，认为要抑制掉 x_t (x_t 的介词、空格可能对我没那么重要)，什么时候改变什么时候不改变通过另外一个带有权重的东西来学习（用另外一组权重来学习是不是要抑制掉当前的输入）

——忘掉过去：很久信息和现在没有太多的相关，把他忘掉

RNN and Gated RNN

The only diff to MLP

- Simple RNN: $\mathbf{h}_t = \phi(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{hx}\mathbf{x}_t + \mathbf{b}_h)$



- Gated RNN (LSTM, GRU): finer control of information flow
 - Forget input: suppress $\mathbf{x}_t$ when computing $\mathbf{h}_t$
 - Forget past: suppress $\mathbf{h}_{t-1}$ when computing $\mathbf{h}_t$

最简单 rnn 的实现

inputs 是一个三维的东西，是一组输入

output 对于每一个时间的样本输出

Code

- Implement Simple RNN

```
W_xh = nn.Parameter(torch.randn(num_inputs, num_hiddens) * 0.01)
W_hh = nn.Parameter(torch.randn(num_hiddens, num_hiddens) * 0.01)
b_h = nn.Parameter(torch.zeros(num_hiddens))

H = torch.zeros(num_hiddens)
outputs = []

for X in inputs: # `inputs` shape : (num_steps, batch_size, num_inputs)
    H = torch.tanh(X @ W_xh + H @ W_hh + b_h)
    outputs.append(H)
```

Full code at http://d2l.ai/chapter_recurrent-neural-networks/rnn-scratch.html

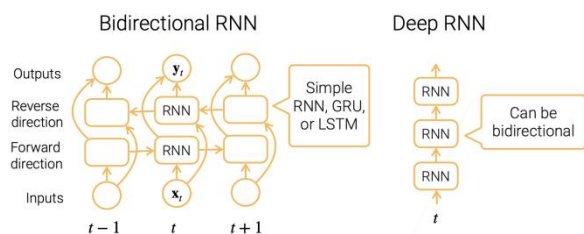
双向 rnn

反向：在时刻 $t+1$ 的输出作为时刻 t 的隐藏状态

$\mathbf{y}_t$ 既有过去方向的信息也有之后方向的信息

多层 rnn

Bi-RNN and Deep RNN



Stanford CS 329P (2021 Fall) - <https://c.d2l.ai/stanford-cs329p>

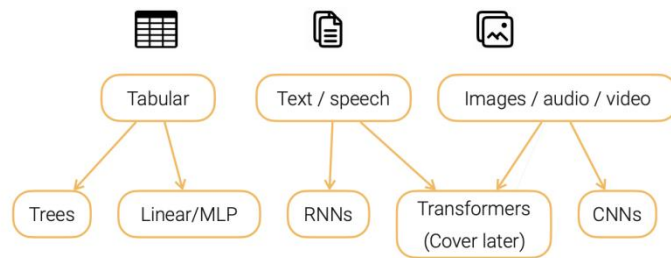
表格数据用树模型、MLP/线性模型

文本有时序信息用 rnn

图片有空间信息用 cnn

变形金刚既能处理时序信息又能处理空间信息，时序是一维，空间是二维，在维度之间有耦合关系，相对来说更加一般化，效果不错

Model Selections



【总结】

多层感知机：将多个全连接层堆起来，中间通过激活函数得到非线性模型

Cnn：卷积层使用空间上的本地性和平移不变性的特性做了个简化版的全连接层，参数更少更适合处理空间信息，卷积层、汇聚层累积起来得到能够有高效抓去空间信息的模型

Rnn:全连接层在时序上把过去的信息放到现在加进来，加了一条额外的边，得到一个循环神经网络，非常适合做有时序信息的数据，通过隐藏状态来处理时序信息

四、模型评估指标

4.1 模型评估

关心模型预测性能如何

怎样衡量模型的好坏

在监督学习里面，通常会最小化训练损失来训练模型

——loss value 通常是被用来衡量一个模型质量的一个很好的指标

一个模型质量通常有多个指标来衡量

-做分类评估模型的精度，做目标检测评估 mAP

-将模型部署到产品里面的时候会有很多商业相关的指标，比如模型对整个营收增长/反应速度

-会看多个指标

例子：指标怎么相互影响——广告怎么变成机器学习问题

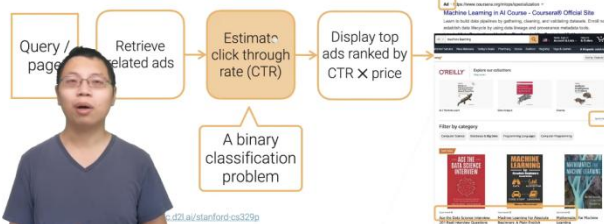
预测广告展现给用户，用户有多大概率去点

Why-对于广告商来说收取的费用是用户点击的次数

用机器学习的手段来判断广告点击还是不点击/点击的概率（二分类问题）

Case Study: Displaying Ads

- Ads is one major revenue source for Internet companies



常见分类指标

-准确度：样本中预测正确类别的个数是多少

广告点击问题是负类（不点击）特别多，正类特别少，不关心对负类的预测，更关心对正类的预测是否正确，对这些不平衡的问题通常会去看每一个类里面的的一个精度和召回率

-精确度：对具体类 i 预测的正确个数/把多少个样本预测成了类 i （分子是预测成了类 i 而且真的是类 i 的个数，分母预测里面有多少个预测成了类 i ）

-召回：对某个类样本里面是多少，对类 i 的话样本里面有多少是类 i

召回和精度分子是一样的，分母不一样。可以把一个东西精度预测的特别高，尽量只会特别置信的时候才会预测成那一类。但这个策略在这个地方会失效，要看到底有多少个真的是正类，召回比较低，要去做精度和召回的权衡

-F1：平衡精度和召回：做一个调和平均

——以上为对于分类来说要看到某一个具体类的时候经常做的指标

Metrics for Binary Classification



- Accuracy: # correct predictions / # examples

```
sum(y == y_hat) / y.size
```

- Precision: # True positive / # (True positive + False positive)

```
sum((y_hat == 1) & (y == 1)) / sum(y_hat == 1)
```

- Recall: # True positive / # Positive examples

```
sum((y_hat == 1) & (y == 1)) / sum(y == 1)
```

- Be careful of division by 0
- One metric that balances precision and recall

- F1: the harmonic mean of precision and recall: $2pr/(p + r)$

Stanford CS 329P (2021 Fall) - <https://c2l.ai/stanford-cs329p>

在广告里面经常用的——AUC&ROC

在广告里面是一个二分类问题，比较关心的是说模型是不是真的能把两个类区分开来，所谓的区分开来可以通过一个东西来衡量：

假设对一个样本的预测值是 $\hat{y}$ （这个样本是正类的概率是 0.9/0.8/0.7），会选一个阈值 θ ， θ 是说假设预测值大于它就认为是正类，小于它是负类（如果正负样本平衡的时候可以取 0.5，但如果样本不那么平衡怎么办。在实际的生产中会去选 θ ，就想把特别之心的那一块选出来，特别关心预测的精度怎么样，就把 θ 调大，只有大于 0.9 才认为是正类，这样提升了精度，但召回率会变低）——怎么看这个模型对调 θ 的反映——看 ROC 曲线

roc 曲线——x 轴分母是有多少个负例的样本，分子是预测成了正类实际是负类

——y 轴是正类的样本数，分子是有多少个真的预测成了正类

ROC 的每一个点表示的是不同的 θ

——表示的是在精度和召回之间的权衡

AUC——ROC 曲线下面的的面积（Auc 的最小值是 0.5，最大值是 1）0.5 特别糟糕，1 特别好

0.7——有些值被耦合在一起，不好区分开来

0.5——正例和负例的预测值基本都是耦合在一起，分布上看起来没什么区别，这个模型没有任何的能力把正例和负例区分开来，基本上是一个随机的预测

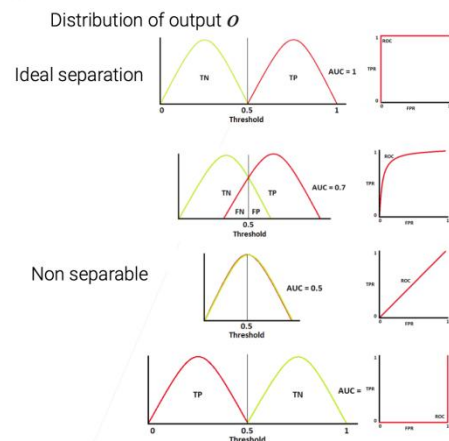
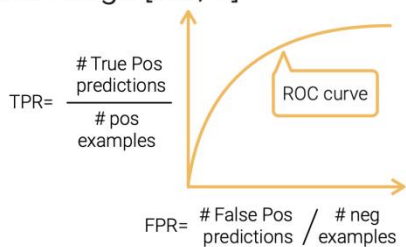
——主要是看模型对于正负类区分的好坏程度，所以经常会被用在广告这个领域，广告是二分类问题，很多时候不关心绝对的召回率/精度，关心的是说这个模型真的能把点击和不点击的广告区分开来

——使得在做产品的时候能够去合理的切那根线，把哪一些广告拿出去。把最置信的广告拿出去很多时候是在优化用户的体验；广告商说收入不行，营收跟不上，就会把 θ （曲线）往回拉，方根多的广告出去

AUC-ROC



- Measures how well the model can separate the two classes
- Choose decision threshold θ , predict positive if $\sigma \geq \theta$ else neg
- In the range [0.5, 1]



把模型部署到产品要考虑的一些商业上的指标

任务：优化收入和用户体验（广告主、用户）

——时效性 latency: 300ms

——ASN: 平均每一页展现多少广告

——CTR: 真实用户点击的概率

——ACP: 每个点击平均下来广告主给平台多少钱

收入=流量（有多少用户来看）*ASN（每个网页出多少广告）*CTR（广告平均被点多少次）

*ACP(每次平均多少钱)

Business Metrics for Displaying Ads

- Optimize both revenue and customer experience
 - Latency: ads should be shown to users at the same time as others
 - ASN: average #ads shown in a page
 - CTR: actual user click through rate
 - ACP: average price advertiser pays per click
- revenue = #pageviews × ASN × CTR × ACP



对指标的权衡

——把训练的模型部署到线上，用实际的数据观察这些商业的指标，如果商业指标达标就可以上线

Displaying Ads: Model → Business Metrics



- The key model metric is AUC
- A new model with increased AUC may harm business metrics, possible reasons:
 - Lower estimated CTR → less ads displayed
 - Lower real CTR because we trained and evaluated on past data
 - Lower prices
- Online experiment: deploy models to evaluate on real traffic data

总结

对一个模型通常会考虑多个指标，如 AUC 看对于正类和负类区分的程度

商业指标评估模型怎么样影响产品，好的模型一定要通过商业验证才能上线

4.2 欠拟合和过拟合

训练误差：模型在训练中能看到的一些错误率

泛化误差：模型在新的没有见过的数据中会怎么样

Underfitting and Overfitting

		Training error	
		Low	High
Generalization error	Low	Good	Bug?
	High	Overfitting	Underfitting

欠拟合：训练误差和泛化误差都高

过拟合：训练误差和泛化误差差距大

考虑数据的复杂性和模型的复杂性

Data and Model Complexity

		Data complexity	
		Low	High
Model complexity	Low	Normal	Underfitting
	High	Overfitting	Normal

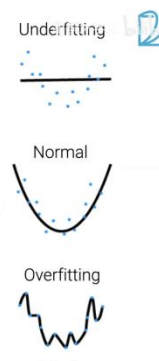


Image credit: hackernoon.com

模型的复杂度

——能够去拟合各种各样函数的能力

——不那么好比较算法之间的复杂度：神经网络 vs 树

——如果是在同一个类里面，可以比较：可学习参数的个数、可以学习的参数里面能取多少值

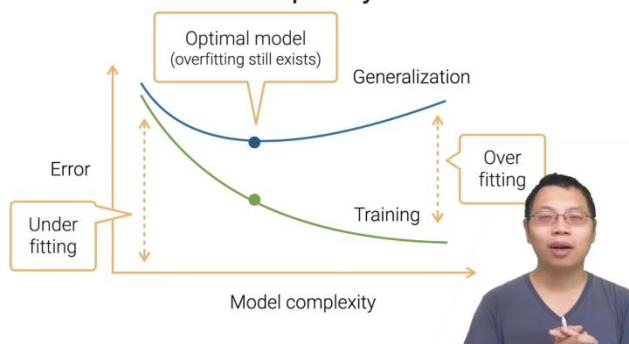
Model Complexity

- The ability to fit variety of functions
 - Low complexity models struggles to fit training set
 - High complexity models can memorize the training set
- It's hard to compare between very different algorithms
 - E.g. trees vs neural network
- In an algorithm family, two factors matter:
 - The number of parameters
 - The values taken by each parameter



模型复杂度会产生的影响

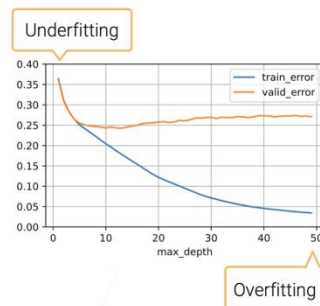
Influence of Model Complexity



Model Complexity Example: Decision Tree



- The tree size can be controlled by the number of levels
- Use scikit-learn
`DecisionTreeRegressor(max_depth=n)` on house sales data



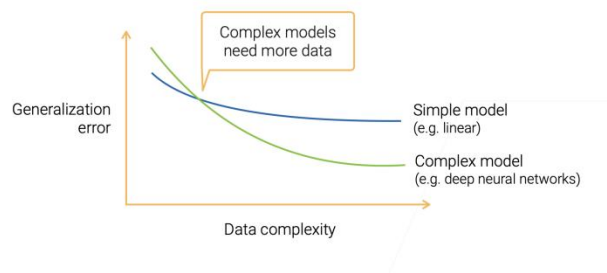
数据复杂度

- 多个样本
- 每个样本里有多少元素
- 时间/空间结构（股票时许、图片）
- 多样性

——很难比较不同类型的数据的复杂度

模型复杂度和数据复杂度都在变换

Model Complexity vs Data Complexity



Generalization error

- Generalization error bound (an informal statement)

$$|\text{error on unseen data} - \text{training error}| \leq \sqrt{\frac{D}{N} \left(\log \left(\frac{2N}{D} \right) + 1 \right)}$$

- D: VC-dim, M: number of training examples
- Generalization error also depends on the training algorithm
 - Adding regularization can penalize complex models
 - Model trained with stochastic gradient methods generalizes better

模型选择

-选择和数据有合适的复杂度的模型

——降低泛化误差

——也要考虑商业指标（如时效性）

-选择一个模型的类，再在里面选择合适的超参数

Model Selection



- Pick a model with a proper complexity for your data
 - Minimize the generalization error
 - Also consider business metrics
- Pick up a model family, then select proper hyper-parameters
 - Trees: #trees, maximal depths
 - Neural networks: architecture, depth (#layers), width (#hidden units), regularizations

总结

关心的是泛化误差，而不是训练误差

模型复杂度：能够拟合各种各样函数的能力

数据复杂度：数据里面有多少信息

模型选择：选择对数据合适的模型

4.3 模型验证 model validation

测试数据集：只能被使用一次

验证数据集：可以被使用多次

常见的集中生成验证数据集的办法

-随机采样分成训练集和验证集

-随机选取 $n\%$ 样本作为验证集

—— 随机行的时候是假设数据是独立同分布的时候 iid

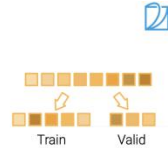
不行如：时序信息（房价、股票）

样本属于不同 group

不平衡数据

Hold Out Validation

- Split your data into "train" and "valid" sets (often calls "test")
- Train your model on the train set, use the error on the valid set to approximate the generalization error
- Often randomly select $n\%$ examples as the valid set
- Typical choices $n = 50, 40, 30, 20, 10$



Split non I.I.D. data

- Random data splitting may lead to underestimate of generalization error
- Sequential data
 - e.g. house sales, stock prices
 - Valid set should not overlap with train set in time
- Examples are highly clustered
 - e.g. photos of the same person, clips of the same video
 - Split clusters instead of examples
- Highly imbalanced label classes
 - Sample more from minor classes

k 则交叉验证

-使用在当数据没有那么多的时候

-把训练数据切成 k 块

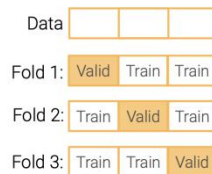
-对 k 中每一次把第 i 个拿出来作为验证集，剩下的作训练

-把平均 k 个误差报告出来做验证误差

-一般来说选 $k=5$ 或 10

K-fold Cross Validation

- Useful when not sufficient data
- Algorithm:
 - Partition the training data into K parts
 - For $i = 1, \dots, K$
 - Use the i -th part as the validation set, the rest for training
 - Report the validation error averaged over K rounds
- Popular choices: $K = 5$ or 10



常见错误

-模型效果特别好-bug 造成，验证集和训练集有一定的重叠

-验证集有来自训练集的样本

-数据融合经常会发生（多个数据源的数据有重合）

-信息泄漏-经常发生在非 iid 数据

Common Mistakes

- If your ML model performance is too good to be true, very likely there is a bug, and contaminated valid set is the #1 reason.
- Valid set has duplicated examples from train set
 - Often happens when integrating multiple datasets
 - Scrape images from search engine to evaluate models trained on ImageNet
- Information leaking from train set to valid set
 - Often happens for non I.I.D. data
 - use future to predict past, see a person's face before
- Excessive use of valid set for hyper param tuning is cheating

Summary



- The test data is used once to evaluate your model
- One can hold out a validation set from the training data to estimate the test data
 - You can use valid set multiple times for model selections and hyper param tuning
 - Validation data should be close to the test data
 - Improper valid set is a common mistake that lead to over estimate of the model performance

五、模型融合

5.1 偏差和方差

——在统计学习里面衡量模型的重要指标

偏差：学习到的模型和真实模型之间的区别-位移

方差：每次学习到的东西的差别多大

低偏差和低方差是比较好的模型

Bias & Variance

- Sample data $D = \{(x_1, y_1), \dots, (x_n, y_n)\}$ from $y = f(x) + \epsilon$
- Learn $\hat{f}_D$ from data D by minimizing MSE: $\min_{\hat{f}_D} \sum_{(x_i, y_i) \in D} (y_i - \hat{f}_D(x_i))^2$
- We want $\hat{f}_D$ generalizes well to an unseen data point (x, y) .

Distribution of $\hat{f}_D(x)$ for different experiment D

Stanford CS 329P (2021 Fall) - <https://c.d2l.ai/stanford-cs329p>

D 是数据集

目标是：采样到的 D 来学习 $\hat{f}$ ，使得 $\hat{f}$ 尽量和 f 相近

——最小化 mse 来学习

Bias-Variance Decomposition



- Learn $\hat{f}_D$ from dataset D sampled from $y = f(x) + \epsilon$
- Evaluate generalization error $(y - \hat{f}_D(x))^2$ on a new data point (x, y)

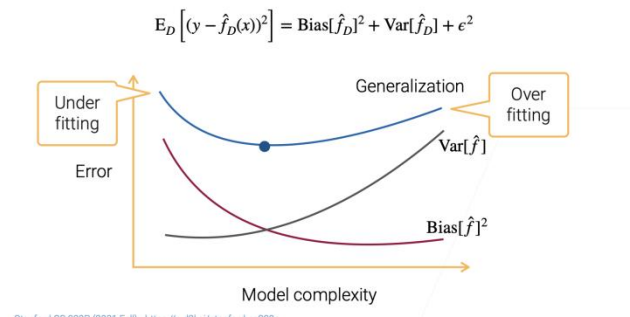
$$\begin{aligned} E_D \left[(y - \hat{f}_D(x))^2 \right] &= E_D \left[\left((f - E_D[\hat{f}_D]) - (\hat{f}_D - E_D[\hat{f}_D]) + \epsilon \right)^2 \right] \\ &= (f - E_D[\hat{f}_D])^2 + E_D \left[(\hat{f}_D - E_D[\hat{f}_D])^2 \right] + \epsilon^2 \\ &= \text{Bias}[\hat{f}_D]^2 + \text{Var}[\hat{f}_D] + \epsilon^2 \end{aligned}$$

交叉项都是独立的被消掉了

所以

——泛化的误差=偏差的平方+方差+数据本身的噪音

Bias-Variance Tradeoff



模型过于简单学不到真实要表达的东西-偏差大，模型越来越复杂，能学到真实的 f 表达的东西——偏差随着模型复杂度递减
过多的看数据里的噪音会导致训练出来的方差变大

任务——最小的泛化误差

减少偏差

-复杂模型

-boosting、stacking

减少方差

-简单点模型

-正则：限制每个模型参数能学习的范围 L1、L2

减少 sigma 平法

-统计学里叫不可以降低的误差，实际在真实场景里来自数据采集的东西，通过更精确更干净的数据来降低

Reduce Bias & Variance



$$E_D \left[(y - \hat{f}_D(x))^2 \right] = \text{Bias}[\hat{f}_D]^2 + \text{Var}[\hat{f}_D] + \epsilon^2$$

- Reduce bias
 - A more complex model
 - e.g. increase #layers, #hidden units of MLP
 - Boosting
 - Stacking
- Reduce variance
 - A simpler model
 - e.g. regularization
 - Bagging
 - Stacking
- Reduce σ^2
 - Improve data

Ensemble learning: train and combine multiple models to improve predictive performance

具体来说，boosting、bagging、stacking 叫集成学习——通过多个模型来提升预测性能

总结

Summary

- Decompose model generalization error into bias, variance and intrinsic error
- Ensemble learning combines multiple models to reduce both bias and variance

5.2 Bagging

独立（并行）训练 n 个模型，在得到 n 个模型，把每个模型自己的输出作平均（回归）/分类做投票

每个模型通过 bootstrap 采样训练出来

——假设有 m 个可以训练的样本（训练集有 m 个样本），每次训练的时候随机采样 m 个样本（不记录放回——虽然还是得到了长为 m 的训练集，但里面有些样本是重复的）

训练 n 个模型，就重复做 n 次

大概含 67% 原始样本，剩下的样本可以作为验证集

Bagging - Bootstrap AGGREGatING

- Learn n base learners in **parallel**, combine to reduce model variance
- Each base learner is trained on a bootstrap sample
 - Given a dataset of m examples, create a sample by randomly sampling m examples with replacement
 - Around $1 - 1/e \approx 63\%$ unique examples will be sampled use the out-of-bag examples for validation
- Combine learners by averaging the outputs (regression) or majority voting (classification)
- Random forest: bagging with decision trees
 - usually select random subset of features for each bootstrap sample

Bagging Code (scikit-learn)

```
class Bagging:
    def __init__(self, base_learner, n_learners):
        self.learners = [clone(base_learner) for _ in range(n_learners)]

    def fit(self, X, y):
        for learner in self.learners:
            examples = np.random.choice(
                np.arange(len(X)), int(len(X)), replace=True)
            learner.fit(X.iloc[examples, :], y.iloc[examples])

    def predict(self, X):
        preds = [learner.predict(X) for learner in self.learners]
        return np.array(preds).mean(axis=0)
```

Clone——每个 baselearner 之间是独立的

随机森林

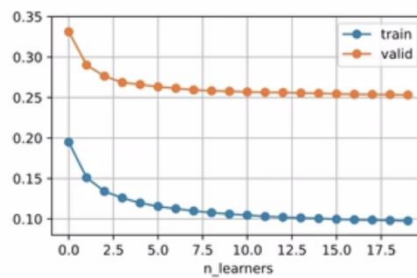
-使用决策树做 base learner

-每次随机采样一些特征（列也随机采样）

好处：随机采样之后能避免一定程度的过拟合，增加每一棵树之间的差异性

Random Forest

- Use decision tree as the base learner
- Often randomly select a subset of features for each learner
- Result on house sale data:



改善了方差，也没有增加偏差，泛化误差没有随着 n 增加往上升

——bagging 主要下降的是方差

采样 1 次和采样 n 次取平均，bias 不会发生变化，唯一下降的是方差

不稳定模型方差下降比较快

Apply bagging with unstable Learners

- Bagging reduces model variance, especially for unstable learners

- Given ground truth f and a set of base learners $\hat{f}_D$
for regression, bagging prediction $\hat{f}(x) = E_D[\hat{f}_D(x)]$

- $E[X]^2 \leq E[X^2]$

$$\left(f(x) - \hat{f}(x)\right)^2 \leq E \left[\left(f(x) - \hat{f}_D(x)\right)^2 \right] \Leftrightarrow \hat{f}(x)^2 \leq E \left[\hat{f}_D(x)^2 \right]$$

With
bagging

Single
learner

Stanford CS 329P (2021 Fall) - <https://c.d2l.ai/stanford-cs329p>

bagging 在 base learner 不那么稳定的时候，对于下降方差的效果更多

——尽量使用不那么稳定的 learner 作为 base learner

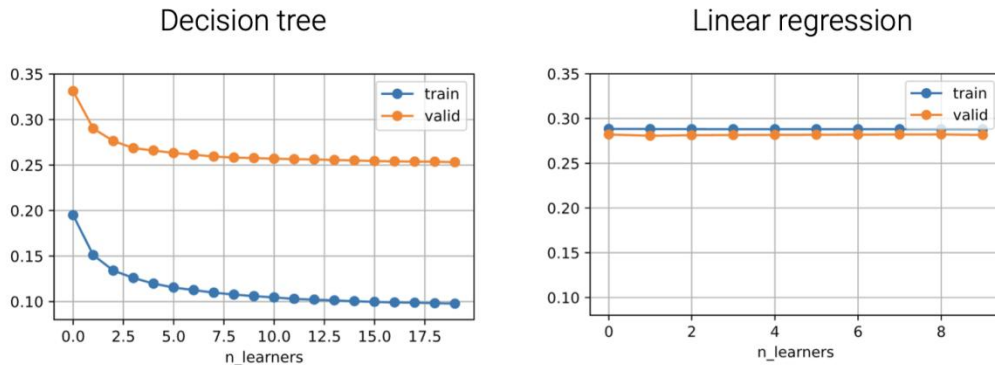
——决策树是一个不那么稳定的模型，数据一旦变化，选取特征做 splitting 就会选的不一样，适合做 bagging

——线性回归比较稳定

Unstable Learners



- Decision tree is unstable, linear regression is stable



总结

bagging 就是训练多个模型，每个模型通过 bootstrap 采样取出来（每一次取出 m 个样本，随机取出 m 个样本，每采样一个把一个放回去）

bagging 主要的效果是降低方差，特别是做决策树效果好

5.3 Boosting

boosting 的想法是将 n 个弱一些模型组合成一个比较强的模型，得到一个偏差比较小的模型

按顺序的学习 n 个模型：

在时间 i 的时候训练一个弱模型 h_i ，根据 h_i 的误差评估一下

下一个模型 h_{i+1} 关注预测不正确的样本

著名算法：AdaBoost、gradient boosting

gradient boosting

Gradient Boosting



- Supports arbitrary differentiable loss
- $H_t(x)$: output of combined model at timestep t , with $H_1(x) = 0$
- For each step t , repeat:
 - Train a new learner $\hat{f}_t$ on residuals: $\{(x_i, y_i - H_t(x_i))\}_{i=1, \dots, m}$
 - Combine: $H_{t+1}(x) = H_t(x) + \eta \hat{f}_t(x)$ shrinkage parameter η for regularization
- MSE $L = \frac{1}{2}(H(x) - y)^2$, residual equals negative gradient $y - H(x) = -\frac{\partial L}{\partial H}$
 - For other loss L , learner $\hat{f}_t = \arg \min \frac{1}{2} \left(\hat{f}_t(x) + \frac{\partial L(x)}{\partial H_t} \right)^2$
- Avoid overfitting: subsampling, shrinkage, early-stopping

Stanford CS 329P (2021 Fall) - <https://c2d.ai/stanford-cs329p>

通过学习率来控制过拟合，不要完全拟合

训练 h_t 去拟合负梯度加上学习率

Gradient Boosting Code



```
class GradientBoosting:
    def __init__(self, base_learner, n_learners, learning_rate):
        self.learners = [clone(base_learner) for _ in range(n_learners)]
        self.lr = learning_rate

    def fit(self, X, y):
        residual = y.copy()
        for learner in self.learners:
            learner.fit(X, residual)
            residual -= self.lr * learner.predict(X)

    def predict(self, X):
        preds = [learner.predict(X) for learner in self.learners]
        return np.array(preds).sum(axis=0) * self.lr
```

Gradient boosting 非常容易过拟合，需要对他做一些正则化，用一个弱的模型，决策树可以是强模型，可以让层数比较少/随机采样特征

GBDT

_决策树层数比较低

-没有过拟合：层数比较低（小模型比较弱）、学习率比较低（0.1）

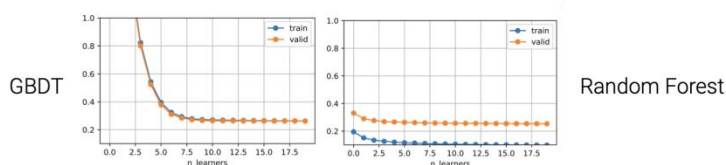
当 weak learner 的学习率控制比较好的时候，过拟合现象不严重（跟 bagging 比）

随机森林每棵树并行训练，boosting 一棵一棵树顺序训练，在比较大的数据集上比较吃亏，xgboost、lightgbm 都使用了一些加速的算法，使得在构建树的时候相对比较快

Gradient Boosting Decision Trees (GBDT)



- Use decision tree as the weak learner
 - Regularize by a small max_depth and randomly sampling features
- Sequentially constructing trees runs slow
 - Popular libraries use accelerated algorithms, e.g. XGBoost, lightGBM



总结

boosting 将 n 个比较弱的模型组合在一起用来降低偏差（bagging 降低的是方差）

Gradient boosting 是 boosting 的一种，每一次弱的模型是去拟合标号上的残差（给定损失函数的负梯度方向）

5.4 Stacking

——刷榜的利器，在竞赛中经常被使用

将多个 baselearner 放在一起用来降低方差

——和 bagging 的区别是 stacking 的 base learner 可以有不同的模型类别

——bagging 通过每次在不同的数据上训练同一个模型，stacking 在同样的数据上训练同样的模型（模型种类不一样，他们差别比较大，以此来降低方差）

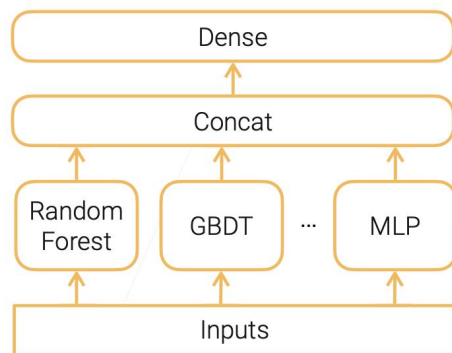
——stacking 的输出是把每个模型的输出连起来，然后再做一个线性的组合得到最终的输出

——加一个全连接层：输入大小是 n ，输出是 1

Stacking



- Combine multiple base learners to reduce variance
 - Base learners can be different model types
 - Linearly combine base learners outputs by learned parameters
- Widely used in competitions
- bagging VS stacking
 - Bagging: bootstrap samples to get diversity
 - Stacking: different types of models extract different features



Stacking Results



- Evaluate on house sales data, compare to bagging and GBDT we implemented before

	Test Error
GBDT	0.259
RandomForest	0.243
Stacking (AutoGluon)	0.229

	model	score_test	score_val	pred_time_test
0	WeightedEnsemble_L2	-0.229626	-0.222406	3.953961
1	ExtraTrees	-0.232468	-0.232027	1.831407
2	CatBoost	-0.238690	-0.230338	0.016279
3	LightGBM	-0.239441	-0.226900	0.203499
4	NeuralNetMXNet	-0.241847	-0.246979	4.397190
5	RandomForest	-0.242904	-0.234931	1.750319
6	XGBoost	-0.249837	-0.240684	0.086715
7	KNeighbors	-0.457131	-0.443591	0.107212

```
from autogluon.tabular import TabularPredictor

predictor = TabularPredictor(label=label).fit(train)
```

上面的 12 是所有的模型加权求和

上面的 stacking 是降低方差，也可以通过多层的 stacking 来降低偏差

长为 n 的向量可以认为是一个特征进入下一层

也可以原始特征+下一层输出再进入下一层

——多层 stacking 特征特别容易过拟合

避免过拟合：

1、训练第一层和第二层用不同的数据——把数据分成 A 和 B 两块

2、重复 k 折 bagging

——1、把一个数据集分成 k 份

——2、把在验证时的输出留下来，并起来——会得到一个和以前数据样本一样多的，但

预测都是没有参与当前模型训练的一个东西

——用上面的样本进行下一层训练

——对 n 个模型用 k 折的办法，训练了 $k \times n$ 个小模型

——进一步降低过拟合——重复 1、2 步骤 n 次，对结果求平均，再把它放到下一层

Multi-layer Stacking Results

- Use 1 additional stacked level, with 5-fold repeated bagging

- Error: 0.229 $\rightarrow$ 0.227

- Training time: 39 sec $\rightarrow$ 207 sec (5x)

```
from autogluon.tabular import TabularPredictor

predictor = TabularPredictor(label=label).fit(
    train, num_stack_levels=1, num_bag_folds=5)
```

Stanford CS 329P (2021 Fall) - <https://c.d2l.ai/stanford-cs329p>



	model	score_test	score_val
0	NeuralNetMXNet_BAG_L2	-0.225332	-0.219718
1	WeightedEnsemble_L3	-0.226921	-0.216254
2	CatBoost_BAG_L2	-0.227525	-0.217471
3	WeightedEnsemble_L2	-0.228386	-0.218298
4	LightGBM_BAG_L2	-0.228400	-0.218374
5	XGBoost_BAG_L2	-0.228660	-0.218824
6	ExtraTrees_BAG_L2	-0.228751	-0.217563
7	ExtraTrees_BAG_L1	-0.233527	-0.224974
8	RandomForest_BAG_L2	-0.234270	-0.220346
9	CatBoost_BAG_L1	-0.237356	-0.227126
10	LightGBM_BAG_L1	-0.238102	-0.225848
11	NeuralNetMXNet_BAG_L1	-0.238413	-0.238786
12	XGBoost_BAG_L1	-0.241698	-0.235570
13	RandomForest_BAG_L1	-0.242029	-0.227800
14	KNeighbors_BAG_L1	-0.457909	-0.447980

总结

stacking 是用多个不一样的模型合并起来来降低方差

多层 stacking 每一层在下一层的基础上再更好的拟合数据来降低偏差——但非常容易过拟合

——通过重复的 k 折 bagging 来降低过拟合

Model Combination Summary



Reduce	Bias	Variance	Computation Cost	Parallelization
Bagging	-	Y	n	n
Boosting	Y	-	n	1
Stacking	-	Y	n	n
K-fold multi-level stacking	Y	Y	$n \times l \times k$	$n \times k$

n : number of learners, l : number of levels, k : k-fold

Stanford CS 329P (2021 Fall) - <https://c.d2l.ai/stanford-cs329p>

六、调参

6.1 模型调参

一般会从好的基线 baseline 开始（所谓的 baseline 就是选一个质量比较高的工具包，工具包里面会设一些还不错的参数，不见得是最好的，但是可以从他开始慢慢调到要的值）

——最好每次调一个值

——重复多次，找规律——炼丹、玄学

Ada 对超参数没那么敏感，SGD 比较敏感

Manual Hyperparameter Tuning

- Start with a good baseline, e.g. default settings in high-quality toolkits, values reported in papers
- Tune a value, retrain the model to see the changes
- Repeat multiple times to gain insights about
 - Which hyperparameters are important
 - How sensitive the model to hyperparameters
 - What are the good ranges



做好日志，管理实验

训练日志和超参数记录下来，如 tensorboard

Manual Hyperparameter Tuning

- Needs careful experiment management
- Save your training logs and hyperparameters to compare, share and reproduce later
 - The simplest way is saving logs in text and put key metrics in Excel
 - Better options exist, e.g. tensorboard and weights & bias
- Reproducing is hard, it relates to
 - Environment (hardware & library)
 - Code
 - Randomness (seed)



随机性——改变随机种子抖动比较大要注意——对付随机性的办法：ensemble

通过工具进行超参数调节

Automated Hyperparameter Tuning



- Computation costs decrease exponentially, while human costs increase
- Cost per training for a typical ML task:
 - E.g. 1M user logs, 10K images
- Cost of a data scientist per day >\$500
- Use algorithms if it outperforms human after 1000 trials
 - Typically beat 90% data scientists

	Time	Cost on cloud
Trees	10min on	\$0.4
Neural networks	1	\$5



自动调参(AutoML)

-HPO 超参数优化

-NAS 构造一个比较好的神经网络模型能够 fit 数据和任务上

Automated Machine Learning (AutoML)



- Automate every step in applying ML to solve real-world problems: data cleaning, feature extraction, model selection...
- **Hyperparameter optimization (HPO):** find a good set of hyperparameters through search algorithms
- **Neural architecture search (NAS):** construct a good neural network model

THE DATA SCIENTIST'S #1 EXCUSE FOR
LEGITIMATELY SLACKING OFF:
"THE AUTOML TOOL IS OPTIMIZING MY MODELS!"



6.2 超参数优化 HPO

在什么样的搜索空间里面选择超参数

Search Space

- Specify range for each hyperparameter

Hyper-Parameter	Range	Distribution
model(backbone)	[mobilenetv2_0.25, mobilenetv3_small, mobilenetv3_large, resnet18_v1b, resnet34_v1b, resnet50_v1b, resnet101_v1b, vgg16_bn, se_resnext50_32x4d, resnext50, resnext200]	categorical
learning rate *	[1e-6, 1e-1]	log-uniform
batch size *	[8, 16, 32, 64, 128, 256, 512]	categorical
momentum **	[0.85, 0.95]	uniform
weight decay **	[1e-6, 1e-2]	log-uniform
detector	[faster-rcnn, ssd, yolo-v3, center-net]	categorical

- The search space can be exponentially large
 - Need to carefully design the space to improve efficiency

Stanford CS 329P (2021 Fall) - <https://c.d2l.ai/stanford-cs329p>

HPO 算法有两大类：black-box or multifidelity

black-box：每一次训练任务当成一个黑盒

——好处：适用于任何机器学习算法

Multifidelity：训练一个任务太贵，不需要训练完整训练集，关心超参数之间的排序，再做一次完整实验

——对数据集进行采样

——减小模型规模

——不需要等到完全训练完，在早期就知道你行不行

HPO algorithms: Black-box or Multi-fidelity



- Black-box: treats a training job as a black-box in HPO:
 - Completes the training process for each trial
- Multi-fidelity: modifies the training job to speed up the search
 - Train on subsampled datasets
 - Reduce model size (e.g less #layers, #channels)
 - Stop bad configuration earlier

HPO algorithms

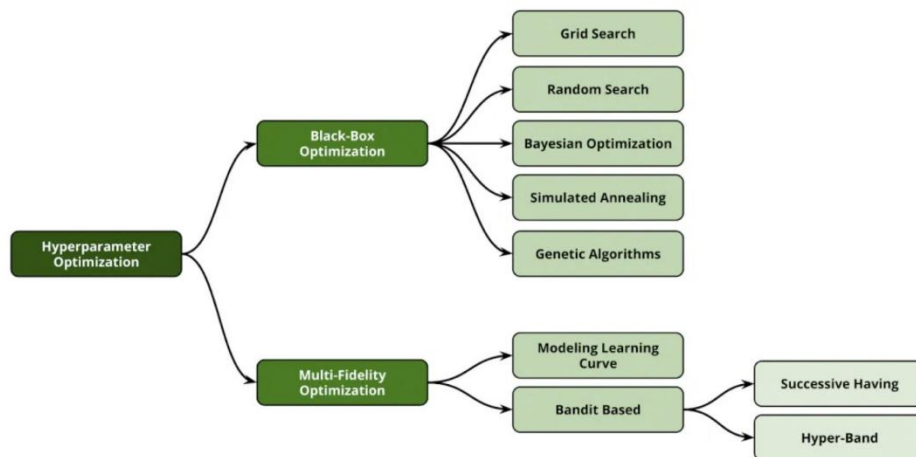


Image credit: [Automated Machine Learning: State-of-The-Art and Open Challenges](#)

Stanford CS 329P (2021 Fall) - <https://c.d2l.ai/stanford-cs329p>

网格搜索

- 所有都覆盖到，保证找到最好的值
- 维度诅咒

随机搜索

- 在 search space 随机选一个 config，只选择最多 n 次
- n 保证了算法不会指数级增加

——一般来说，随机搜索是一个有效的方法，n，一般知道想训练多少小时/有多少资源，训练到超时间就可以停掉，也可以早点停，每次观察随着时间增加，最好的结果是一个增长的曲线，这个曲线每一个点代表当前时刻搜到的最好参数的精度，如果在某个时段变平可以选择停

Two most common HPO strategies

• Grid search

```
for config in search_space:
    train_and_eval(config)
return best_result
```



- All combinations are evaluated
- Guarantees the best results
- Curse of dimensionality

• Random search

```
for _ in range(n):
    config = random_select(search_space)
    train_and_eval(config)
return best_result
```

- Random combinations are tried
- More efficient than grid search (empirically and in theory, shown in [Random Search for Hyper-Parameter Optimization](#))

贝叶斯优化

——active 研究方法，现实中不一定能用到

BO: 学一个从你超参数到目标函数（损失或评估精度都行）的函数，选下一个超参数的時候会根据当前的评估

——机器学习学到是数据到 label/预测值之间的一个 map

里面有两个模型

Surrogate model: 拟合超参数到目标值的模型 (可以用概率的 regression 模型-random forest、高斯过程)

y 轴是目标函数, 希望越高越好。黑色实线表示真实的值, 虚线表示预测值的均值。紫色那块是置信度, 虚线和实线之间表示模型拟合不那么好的地方

——随着采用点越多, 对整个模型建模会越来越准

——接下来考虑下一个点采样在什么地方

——在 acquisition max 那里

获取函数:

对每一个超参数预测一个值, 值大表示不那么置信 (很置信就不用管你了), 拟合不那么置信的区域

采样获取函数值最大的点

——探索和利用的权衡

评价这个算法

——一开始和随机搜索差不多, 后期比随机搜索好一些

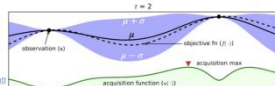
——整体实际用随机搜索多

Bayesian Optimization (BO)

- **BO:** Iteratively learn a mapping from HP to objective function. Based on previous trials. Select the next trial based on the current estimation.

- **Surrogate model**

- Estimate how the objective function depends on HP
- Probabilistic regression models: Random forest, Gaussian process, ...



Stanford CS 329P (2021 Fall) - 101



Bayesian Optimization (BO)

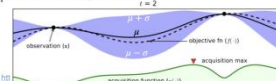


- **Acquisition function**

- Acquisition max means uncertainty and predicted objective are high.
- Sample the next trial according to the acquisition function
- Trade off exploration and exploitation

- **Limitation of BO:**

- In the initial stages, similar to random search
- Optimization process is sequential



Stanford CS 329P (2021 Fall) - 101

Successive Halving

核心是有很多超参数的选择, 大部分超参数没必要把它训练完, 只把最靠谱的那些超参数训练的足够长就行, 剩下的基本早起淘汰掉

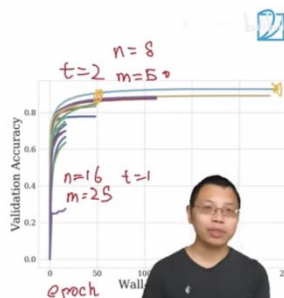
——选取 n 个超参数, 每个超参数训练 m 个 epochs (一开始 n 大一些, m 小一些)

——图 x 轴是 epoch, y 轴是关心的验证精度, $n=16$, $m=25$ ——第一次完评估了 16 组超参数, 每一组扫了 25 遍数据, 这一遍完成之后, 把剩下的一半留下, $t=2$ (第 2 次迭代), $n=8$, $m=50$, 在下一个时刻, $n=4$, $m=100$

——问题: n 和 m 不好取

Successive Halving

- Save the budget for most promising config
- Randomly pick n configurations to train m epochs
- Repeat until one configuration left:
 - Keep the **best** $n/2$ configuration to train another m epochs
 - Keep the **best** $n/4$ configuration to train another $2m$ epochs
 - ...
- Select n and m based on training budget and #epoch needed for a full training



Stanford CS 329P (2021 Fall) - <https://csl.stanford.edu/courses/CS329P/>

Hyperband

——跑多次 sh，每次用不同的 n 和 m

n 是 exploration, m 是 exploitation

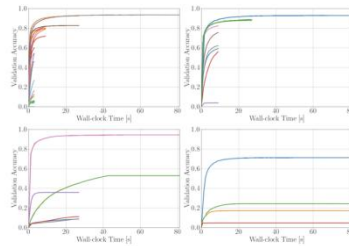
好处：对 m 和 n 选取不那么敏感

——会多跑几次，多几个组合

Hyperband



- In Successive Halving
 - n : exploration
 - m : exploitation
- Hyperband runs multiple Successive Halving, each time decreases n and increases m
- More exploration first, then do more exploit



【总结】

黑盒 HPO：一个超参数进去，一个模型出来指导模型的好坏——这里面有暴力搜索/随机搜索，贝叶斯优化

Multi-fidelity HPO：sh, hyperband

在实际中，从随机搜索开始

注意 top performers

Summary



- Black-box HPO: grid/random search, bayesian optimization
- Multi-fidelity HPO: Successive Halving, Hyperband
- In practice, start with random search
- Beware there are top performers
 - You can find them by mining your training logs, or what configurations used in paper/code



Stanford CS 329P (2021 Fall) - <https://cs.d2l.ai/stanford-cs329p>

6.3 网络架构搜索 (NAS algorithms)

超参数

层

每一层：核窗口大小、输出通道数、隐藏单元的个数

NAS：尽量使得整个网络的设计能够自动化

——关心：

- 1、整个搜索的空间
- 2、怎么样在搜索空间搜索
- 3、怎么样判断神经网络的好坏

Neural Architecture Search (NAS)



- A neural network has different types of hyperparameters:
 - Topological structure: resnet-ish, mobilenet-ish, #layers
 - Individual layers: kernel_size, #channels in convolutional layer, #hidden_outputs in dense/recurrent layers
- NAS automates the design of neural network
 - How to specify the search space of NN
 - How to explore the search space
 - Performance estimation



Image source: [Elsken, et al. 2019](#)

通过强化学习做 NAS

用 RNN 生成网络架构，生成出来之后在数据集上训练得到验证的精度，再把精度返回给它

——好处：通用框架

——坏处：非常贵

NAS with Reinforcement Learning



- [Zoph & Le 2017](#)
 - A RL-based controller (REINFORCE) for proposing architecture.
 - RNN controller outputs a sequence of tokens to config the model architecture.
 - Reward is the accuracy of a sampled model at convergence
- Naive approach is expensive and sample inefficient (~2000 GPU days). To speed up NAS:
 - Estimate performance
 - Parameter sharing (e.g. EAS, ENAS)

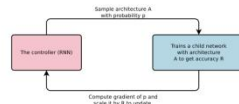


Image source: [Zoph & Le 2017](#)

One-shot 的办法

学习模型的架构和参数

——训练一个特别大的模型，每个子模型就是要考虑的架构，既得到它的性能又得到它的参数

一般来说：用近四指标训练一些数据看一下他们的精度，把最好的选出来再从头开始训练

The One-shot Approach



- Combines the learning of architecture and model params
- Construct and train a single model presents a wide variety of architectures
- Evaluate candidate architectures
 - Only care about the candidate ranking
 - Use a proxy metric: the accuracy after a few epochs
- Re-train the most promising candidate from scratch

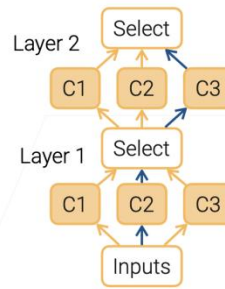
其中一个方法：可微的架构搜索

Differentiable Architecture Search



- Relax the categorical choice to a softmax over possible operations:
 - Multiple candidates for each layer
 - Output of i -th candidate at layer l is o_i^l
 - Learn mixing weights $\mathbf{a}^l$. The input for $i+1$ -th layer is $\sum_i \alpha_i^l o_i^l$ with $\alpha^l = \text{softmax}(\mathbf{a}^l)$
 - Choose candidate $\text{argmax}_i \alpha_i$
 - Jointly learn $\mathbf{a}^l$ and network parameters
- A more sophisticated version (DARTS) achieves SOTA and reduces the search time to ~ 3 GPU days

Stanford CS 329P (2021 Fall) - <https://c.d2l.ai/stanford-cs329p>



加 select 的操作子

——进入下一层尽量让 α_2 权重比较大，趋向于 1（完成了 1 次选择的操作）
（训练完之后做选择的时候每次把候选最大参数拿出来，剩下去掉）

Scaling cnns

——看的是卷积神经网络

调节卷积神经网络的 3 种方式：

Deeper: 更深

Wider: 每层的输出通道

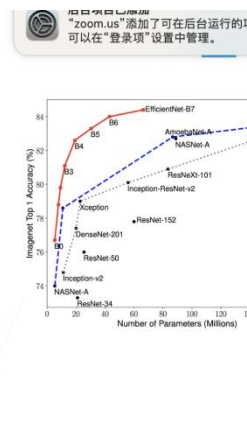
Larger inputs: 让输入变得更大（分辨率更高）

Efficientnet 这三个东西要一起跟着动

Scaling CNNs

- A CNN can be scaled by 3 ways:
 - Deeper: more layers
 - Wider: more output channels
 - Larger inputs: increase input image resolutions
- EfficientNet proposes a compound scaling
 - Scale depth by α^ϕ , width by β^ϕ , resolution by γ^ϕ
 - $\alpha\beta^2\gamma^2 \approx 2$ so increase FLOP by 2x if $\phi = 1$
 - Tune $\alpha, \beta, \gamma, \phi$

Stanford CS 329P (2021 Fall) - <https://c.d2l.ai/stanford-cs329p>



Research directions



- Explainability of NAS result
- Search architecture to fit into edge devices
 - Edge devices are more and more powerful, data privacy concerns
 - But they are very diverse (CPU/GPU/DSP, 100x performance difference) and have power constraints
 - Minimize both model loss and hardware latency
 - E.g. minimize loss $\times \log(\text{latency})^\beta$
- To what extent can we automate the entire ML workflow?

总结

NAS 是搜索神经网络架构，可以有一个定制化目标，比如说最大化精度，满足延迟需求在一个特定硬件上面

NAS 现在用的比较多：

-把卷积神经网络输入的深度、宽度、分辨率合起来调

-可以导到 one-hot 里训练一个特别大的架构

七、深度神经网络共用设计模式

批量和层的归一化

残差连接

https://c.d2l.ai/stanford-cs329p/_static/pdfs/cs329p_slides_13_2.pdf

注意力机制

https://c.d2l.ai/stanford-cs329p/_static/pdfs/cs329p_slides_13_3.pdf

Deep Network Tuning



- DL is a programming language to extract information from data
 - Some values will be filled by data later
 - Differentiable
- Various design patterns, from layers to network architecture
- Here we talk about some of them



7.1 批量和层归一化

【批量和层归一化】

批量归一化：

-线性模型对数据进行标准化（使得数据的每一个特征均值为 0，方差为 1）

- » 从数学上来说，这样可以使损失函数更加平滑
- » 假设 x 和 y 之间隔得不远，梯度改变的方向可以被上限（傍得住）
- » 贝塔比较小的时候可以允许比较大的学习率
- » 不会帮助深度神经网络

批量归一化——» 把中间那些层标准化

第一步：变形

——» 把输入变成 2D 的矩阵

c 在卷积神经网络对应的是一个特征（通道是一个特征，不同神经网络有不同的输出通道，每个通道去识别不一样的模式）

n 批量是一个样本

高宽里面有很多像素、每个像素里面的每个点可以认为是小的数据

——» 变成一个二维矩阵：第一维度是样本，第二维度是特征

第二步：对每一列进行标准化

第三步：还原-希望有偏差

第四步：把 Y' 重新转回以前的格式

Batch Normalization

- **Reshape** input $\mathbf{X}$ into 2D (no change for 2D input $\mathbf{X} \in \mathbb{R}^{n \times p}$)
 - $\mathbf{X} \in \mathbb{R}^{n \times c \times w \times h} \rightarrow \mathbf{X}' \in \mathbb{R}^{n \times wh \times c}$ (batch n , channel c , width w , height h)
- **Normalize** by standardization each column $\mathbf{x}'_j, j = 1, \dots, p$
 - $\hat{\mathbf{x}}'_j \leftarrow (\mathbf{x}'_j - \text{mean}(\mathbf{x}'_j)) / \text{std}(\mathbf{x}'_j)$
- **Recovery** $\mathbf{Y}'$ with $\mathbf{y}'_j = \gamma_j \hat{\mathbf{x}}'_j + \beta_j$ as the j -th column, γ_j, β_j are parameters
- Output $\mathbf{Y}$ by reshaping $\mathbf{Y}'$ to the same shape as $\mathbf{X}$

Batch Normalization Code

```
def batch_norm(X, gamma, beta, moving_mean, moving_var, eps, momentum):
    if not torch.is_grad_enabled(): # In prediction mode
        X_hat = (X - moving_mean) / torch.sqrt(moving_var + eps)
    else:
        assert len(X.shape) in (2, 4)
        if len(X.shape) == 2:
            mean = X.mean(dim=0)
            var = ((X - mean)**2).mean(dim=0)
        else:
            mean = X.mean(dim=(0, 2, 3), keepdim=True)
            var = ((X - mean)**2).mean(dim=(0, 2, 3), keepdim=True)
        X_hat = (X - mean) / torch.sqrt(var + eps)
        moving_mean = momentum * moving_mean + (1.0 - momentum) * mean
        moving_var = momentum * moving_var + (1.0 - momentum) * var
    Y = gamma * X_hat + beta
    return Y, moving_mean, moving_var
```

Full code: http://d2l.ai/chapter_convolutional-modern/batch-norm.html

Standard PS 9500 (2021 Fall) - <https://nlp.seas.harvard.edu/2021/08/13/batch-normalization.html>

层归一化

——» 主要用途：用在循环神经网络里面

批量归一化的问题是：每一个时间步骤用自己的均值方差，和学到的 γ , β 在不同时间步之间不要共享均值和方差
 ——层归一化和批量归一化的区别在于按哪个维度去算均值和方差
 批量归一化：对特征那一列算均值、方差
 层归一化：对每一个样本，按行来做

$N \times p \times t$ ，把 p 放在一个维度

——好处：在做预测的时候，不需要存全局的东西，只需要把每个样本里面的东西算一下，不用看前面样本的均值、方差

Layer Normalization



- If apply to RNN, BN needs maintain separated moving statistics for each time step
 - Problematic for very long sequences during inference
- Layer normalization reshapes input $\mathbf{X} \in \mathbb{R}^{n \times p} \rightarrow \mathbf{X}' \in \mathbb{R}^{p \times n}$ or $\mathbf{X} \in \mathbb{R}^{n \times c \times w \times h} \rightarrow \mathbf{X}' \in \mathbb{R}^{c \times w \times h \times n}$, rest is same with BN
 - Normalizing within each example, up to current time step
 - Consistent between training and inference
 - Popularized by Transformers

别的 normalization

改变变形

- 1、对每个位置在不同样本和通道里面的均值为 0，方差为 1
- 2、把 c 拆开，拆成 n 个组
- 3、把均值、方差算出来之后在不同特征之间交换

改变标准化步骤：

白化——不仅是均值为 0，方差为 1，而且使每个特征之间没有关系（做 PCA）

改变还原的步骤：

变成一个线性层

对权重/梯度做标准化

More Normalizations



- Modify "reshape", e.g.
 - InstanceNorm: $n \times c \times w \times h \rightarrow wh \times cn$
 - GroupNorm: $n \times c \times w \times h \rightarrow sw \times h \times gn$ with $c = sg$
 - CrossNorm: swap mean/std between a pair of features
- Modify "normalize": e.g. whitening
- Modify "recovery": e.g. replace γ, β with a dense layer
- Apply to weights or gradients

总结

normalization 把中间的层数值上更加稳定，损失函数更加平滑，可以使整个神经网络的训练更加容易，一般来说，不会改变最后的那一个精度，但主要是让训练更容易，使得优化的曲线更加平滑，不用他很多时候也是没关系的，可以通过别的途径

一般来说，normalization 干的是以下三个步骤

- 把输入换成合适的形式
- 把列进行标准化和归一化
- 再做一次还原步骤（想要一个不一样的方差、不一样的均值，还原是可以学习的，调成回去的形状）

——批量归一化在计算实现上很多时候会带来很大问题，能够去掉他当然是最好的了

7.2 残差连接

https://c.d2l.ai/stanford-cs329p/_static/pdfs/cs329p_slides_13_2.pdf

7.3 注意力机制

https://c.d2l.ai/stanford-cs329p/_static/pdfs/cs329p_slides_13_3.pdf

八、迁移学习

8.1 迁移学习

迁移学习的作用

- 在一个任务上学习一个模型，可以解决相关的别的任务
- 迁移学习的出圈在深度学习

方法

- 特征抽取
- 在一个相关任务里训练一个模型，在另外一个人任务里直接用它
- 做微调

相关领域

- 半监督学习
- zero-shot/few-shot
- 多任务学习

Transfer learning

- Motivation
 - Exploit a model trained on one task for a related task
 - Popular in deep learning as DNNs are data hungry and training cost is high
- Approaches
 - Feature extraction (e.g. Word2Vec, ResNet-50 feature, I3D feature)
 - Train a model on a related task and reuse it
 - Fine-tuning from a pertained model (focus of this lecture)
- Related to
 - Semi-supervised learning
 - In the extreme, zero-shot / few-shot learning
 - Multi-task learning, where some labeled data is available for each task

8.2 微调在计算机视觉的应用 (fine-tuning in cv)

有多种转移知识的办法

1、用预训练好的模型

神经网络可以把它分成两块

第一块：编码器 encoder

-特征提取器

——把原始的像素转换成线性可分的特征

第二块：解码器 decoder

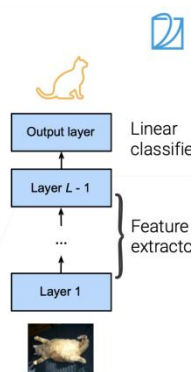
——简单的线性分类器。把编码器的表示映射成标号/做些决策

预训练模型

——在比较大的数据集上训练好的模型，认为有一定的泛化模型

Pre-trained Models

- Partition a neural network into:
 - A feature extractor (encoder) maps raw pixels into linearly separable features
 - A linear classifier (decode) makes decisions
- Pre-trained model
 - a neural network trained on a large-scale and general enough dataset
 - The feature extractor may generalize well to
 - other datasets (e.g. medical/satellite images)
 - other tasks (e.g. object detection, segmentation)



把 pre-trained 好的 model 用到新任务来

——叫 fine tuning 微调

——» 在新的任务上面再构建一个新的模型，新的模型架构和预训练好的模型一样

初始化模型

-权重初始的时候不是用的随机，而是直接把预训练好的模型权重直接复制过来

-最后一层解码器用的是随机

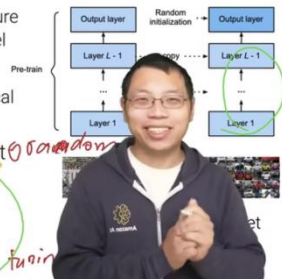
-已经在最优解附近不要走太远，用小一些的学习率（随机用 0.1，微调用 0.001 或更小都 ok）

&&不用训练太长时间——让整个搜索空间变得小一点，不要搜太大——让泛化能力更好，

不要改的太多

Fine-Tuning techniques

- Initialize the new model:
 - Initialize the feature extractor with the feature extractor parameters of a pre-trained model
 - Randomly initialize the output layer
 - Start the parameter optimization near a local minimal
- Train with a small learning rate with just a few epochs
- Regularize the search space



把最底层的那些层固定住

——因为神经网络通常觉得有层次化的学习过程

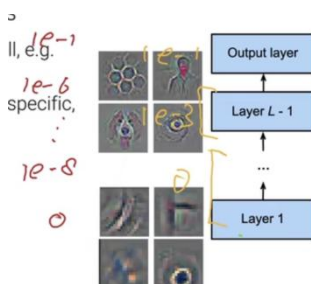
——在最底层通常是学到一些底层的特征，圆圈、颜色的边，随着层数变得越来越高高的时候，多多少少会学到一些更全局的东西，比如学到的是个人，是个狗的脸

——一般来说，下面那些层学到的是像素的底层特征和上面那些层没有太多的关系，换一些数据集泛化性能也是比较好的，越往上越向标号的语义空间靠近

最后一层随机初始化学习

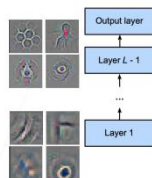
下面对某一层进行改动

最下面那些层微调的时候不动（学习率为0）



Freeze Bottom Layers

- Neural networks learn hierarchical features
 - Low-level features are universal, generalize well, e.g. curves / edges / blobs
 - High-level features are more task and dataset specific, e.g. classification labels
- Freeze bottom layers during fine tuning
 - Train the top layers from scratch
 - Keep low-level universal features intact
 - Focus on learning task specific features
 - A strong regularizer



怎么找到预训练模型

Tensorflow hub <https://tfhub.dev/>

TIMM: <https://github.com/rwightman/pytorch-image-models>

Where to Find Pre-trained Models



- Tensorflow Hub: <https://tfhub.dev/>
- Tensorflow models submitted by users
- TIMM: <https://github.com/rwightman/pytorch-image-models>
- PyTorch models collected by Ross Wightman

```
import timm
from torch import nn

model = timm.create_model('resnet18', pretrained=True)
model.fc = nn.Linear(model.fc.in_features, n_classes)
# Train model as a normal training job
```

微调应用

-微调加速了收敛

-但不一定会提升精度

Applications



- Fine-tuning pre-trained models (on ImageNet) is widely used in various CV applications:
 - Detection/segmentation (similar images but different targets)
 - Medical/satellite images (same task but very different images)
- Fine-tuning accelerates convergence
- Though not always improve accuracy
 - Training from scratch could get a similar accuracy, especially when the target dataset is also large



Stanford CS 329P (2021 Fall) - <https://c.d2l.ai/stanford-cs329p>

总结

-通常会在比较大的训练集上训练预训练好的模型（如图片分类）

-在新的任务上把模型的权重初始成预训练好的模型的权重

-微调通常会加速收敛，有时会提升精度

8.3 微调在自然语言处理的应用（fine-tuning in NLP）

nlp 和 cv 有一个不一样的地方：

nlp 不存在一个应用有一个特别大的标好的数据集

-在 cv 里面可以在图片分类上使用比较大的标注的数据集训练预训练好的模型

-在 nlp 里面有大量的没有标号的文档

-在 nlp 里面一般会去做自监督的预训练（标号是自己产生的伪标号，使用这些伪标号再用监督学习任务来完成预训练）

常见的任务：

语言模型 LM：给你一个句子的前一些词，预测下一些词

带源码的语言模型 MLM：完形填空

Self-supervised pre-training

- No large-scale labeled NLP dataset
- Large quantities of unlabeled documents
 - Wikipedia, ebooks, crawled webpages
- Self-supervised pre-training
 - Generate "pseudo label" and use supervised learning task
 - Common tasks for NLP
 - Language model (LM): predict next word. e.g. I like your **hat**
 - Masked language model (MLM): random masked word prediction. e.g. I like **your** hat

常见的预训练模型

第一类模型

-词嵌入：对每个词学习 uw 和 vw ，任务是带源码的语言模型

左边到右边的词记为 $x_1 \dots x_n$ ， n 是窗口的大小，给定他之后去预测 y

——对于要预测的词 y 用 u 这个向量表示，对上小文的词用 v 来表示

——把上下文的词的特征向量加起来，再跟 y 对应 u 这个向量做内积，这样就得到了 y 这个词和上下文的关系，然后再做预测的时候在所有的词典里面选取一个 y ，使得最大化这个值

——这个模型叫 CBOW

第二类模型

-基于 transformer 的预训练模型（最近应用的比较多）

BERT: transformer 的编码器，使用的是带掩码的词预测，编码在处理的时候会看到完整的句子。双向的

GPT: 用的是解码器，从左到右的过程。可以用来预测下一个词

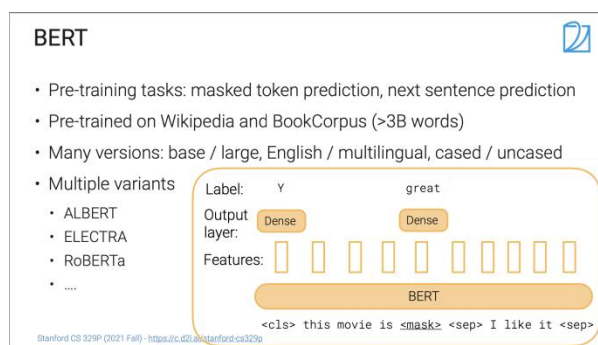
T5: 基于编码器和解码器的架构

BERT

-进入 bert 之后每一个词会得到它向量的表示

-第一个词的向量表示主要是对他进行分类用的（两个句子是相邻关系，在做分类的时候标号是 1，没有相邻关系的话标号是 0）

-要去预测缺失了什么东西的地方会加一个全连接层，来预测真实变化 great



BERT 微调

-和 cv 微调很像

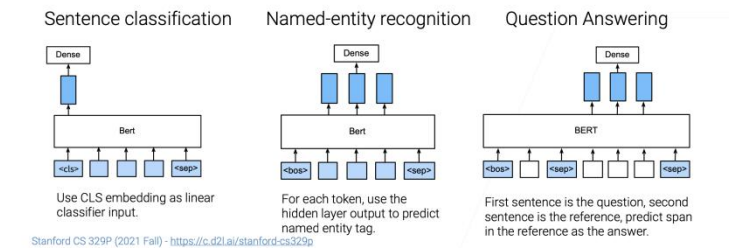
句子分类-把第一个词对应的输出拿出来加输出层

词级别-把每个词对应的向量拿出来，做全连接层

问答-进来是两句话，对第二个句子每个词的输出里面去看一下是不是答案的开始还是不是答案的结束

BERT Fine-tuning

- Randomly initialize the last layer, train a few epochs with small lr
- Downstream task examples



实际考虑

- bert 微调在结果上不稳定
 - 在做微调时用完整版 Adam
 - 多训练几个 epochs

-在 bert 里面可以把上面那些层随机初始化

Practical Considerations

- BERT fine-tuning on small datasets can be unstable
 - BERT removed bias correction steps in Adam
 - Too few (=3) epochs
- Randomly initializing some top transformer layers help
 - Features learned by top layers are too specific to the pre-training tasks
 - The cutoff depends on downstream tasks

Where to Find Pre-trained Models

- HuggingFace: a collection of pre-trained transformer models on both PyTorch and TensorFlow

```
from transformers import AutoTokenizer
from transformers import AutoModelForSequenceClassification

tokenizer = AutoTokenizer.from_pretrained("bert-base-cased")
inputs = tokenizer(sentences, padding="max_length", truncation=True)
model = AutoModelForSequenceClassification.from_pretrained(
    "bert-base-cased", num_labels=2)
# Train model on inputs as a normal training job
```

Stanford CS 329P (2021 Fall) - <https://c.d2l.ai/stanford-cs329p>

要把怎么词元化拿过来 tokenizer
做什么任务要拿不一样的东西

bert 应用

判断句子是否语法正确

电影评论正面/负面
两句话语义是否等价
假设和结论是否有加强关系
在问题中寻找答案的范围

——bert 采用编码器架构有一定的局限性，给你一段话，生成一段新的话，gpt 更适合
T5 做文章 summarization, q&a

Applications



- “(BERT) obtains new state-of-the-art results on eleven natural language processing tasks”, including
 - If a sequence of words is a grammatical English sentence
 - Sentiment of sentences from movie reviews
 - Sentences/questions in a pair are semantically equivalent, or similar
 - If the premise entails the hypothesis
 - Find the span of the answer for a question
- “(T5) achieve state-of-the-art results on many benchmarks covering summarization, question answering, text classification, and more”

总结

在自然语言里面预训练模型通常是通过自监督完成

Bert 是个特别大的 tranformer 编码器

GPT 是个特别大的解码器

T5 编码器+解码器